

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 3

CSS-Grundlagen

Dieses Kapitel erläutert die Grundlagen von Cascading Style Sheets. Von einer Einführung über die grundsätzliche Funktionsweise bis hin zu einigen exemplarischen Formaten. Letztere werden nicht im Detail besprochen, da sie einfach nachgeschlagen werden können.

3.1 Einführung

Der folgende Abschnitt stellt eine grundlegende Einführung in *Cascading Style Sheets* (kurz CSS) dar.

3.1.1 Definition

Cascading Style Sheets sind eine deklarative Sprache zur Definition von Formateigenschaften einer Website. Sie stellen eine unmittelbare Ergänzung zu HTML dar, können aber auch mit anderen Markup-Sprachen wie z. B. XML verwendet werden.

3.1.2 Wozu CSS?

In Vorlesung 2 wurde bereits auf die Wichtigkeit der semantisch korrekten Verwendung von HTML-Elementen hingewiesen. So spezifiziert eine Auszeichnung mit dem `<em>`-Element etwa, dass eine Textstelle betont ist. Um dies visuell darzustellen, stellen Webbrowser die markierte Stelle standardmäßig kursiv dar. Auch alle anderen Elemente stellen Browser in einer bestimmten Art und Weise dar. HTML bietet jedoch keine Möglichkeit (mehr¹), diese vom Browser vorgegebene Darstellung anzupassen. Dies ist auch gut so, denn die Aufgabe von HTML ist es lediglich, Inhalte mit einer Bedeutung zu versehen. Die Darstellung spielt per Definition keine Rolle.

Diese Aufgabe übernehmen die Cascading Style Sheets. Mit ihrer Hilfe kann das Aussehen eines jeden einzelnen HTML-Elements bestimmt bzw. verändert werden, ohne dabei die Semantik zu beeinflussen.

Zusammengefasst gilt:

¹Bis HTML 4.01 gab es noch Elemente und Attribute, die das Aussehen des Dokuments beeinflussen konnten, etwa das Attribut `color`.

In HTML sollen ausschließlich Inhalt und Struktur definiert werden. Cascading Style Sheets dienen zur Definition der Darstellung.

3.1.3 Versionen

Die Entwicklung von Stylesheets begann Anfang der 1990er Jahre, 1995 wurde diese Entwicklung vom W3C berücksichtigt, das damit begann, an der Standardisierung zu arbeiten. Im Dezember 1996 wurde die *CSS Level 1 Recommendation*, also CSS 1.0 [13] veröffentlicht.

CSS Level 2, also CSS2, wurde Mitte 1998 veröffentlicht. Die Umsetzung in den Browsern erfolgte jedoch nur schleppend und auch heute implementiert kaum ein Browser diese Version vollständig. Um auf die Probleme, die mit CSS2 auftraten, zu reagieren, wurde 2002 mit der Entwicklung von CSS Level 2 Revision 1 (kurz CSS 2.1 [11]) begonnen. Diese Version ist die in den letzten Jahren meist benutzte und wurde meist als Standard angesehen. Tatsächlich wurde sie vom W3C erst am 7. Juni 2011 als Recommendation verabschiedet. Die Implementierung ist in allen Browsern sehr gut. Im April 2016 wurde die Arbeit an CSS 2.2 [12] gestartet, welches zum Ziel hat, Fehler aus CSS 2.1 zu beseitigen und auch Features zu entfernen, die nie implementiert oder durch CSS3-Module ersetzt wurden.

Die Entwicklung von *CSS Level 3* (kurz CSS3) startete bereits im Jahr 2000 mit dem Ziel, neue Features zu entwickeln (CSS 2.1 war nur eine Fehlerkorrektur von CSS2 und bot daher nichts Neues). Die Spezifikation ist keine Neuentwicklung von Beginn an, sondern baut im Kern auf CSS 2.1 auf. Neue Entwicklungen werden nicht mehr monolithisch in einem Dokument zusammengefasst, sondern in Form von einzelnen Modulen spezifiziert. Diese werden einzeln entwickelt und befinden sich daher in unterschiedlichen Standardisierungsstufen. CSS3 Module sind in einigen Fällen Ergänzungen in anderen Fällen aber auch Ersatz von CSS 2.1 Eigenschaften. Das W3C veröffentlicht in regelmäßigen Abständen den so genannten *CSS Snapshot* [22], der den aktuellen Status von CSS zusammenfasst. Darin u. a. folgende CSS-Module (aus Ebene 3 bis 5) aufgelistet:

- **CSS 2.1** [11]: Kern der CSS-Spezifikation, befindet sich im Status einer Recommendation (ist also abgeschlossen).
- **CSS Syntax Level 3** [23]: Ergänzungen zur Syntax gegenüber CSS 2.1. Seit 2021 im Status eines Candidate Recommendation Drafts.
- **Media Queries Level 3** [27]: Anpassungen von Layouts, je nach verwendetem Ausgabegerät, seit 5. April 2022 im Status einer Recommendation.
- **Selectors Level 3** [28]: Erweitert die in CSS 2.1 definierten Selektoren um viele neue. Seit November 2018 eine Recommendation.
- **CSS Box Model Level 3** [16]: Ergänzungen und Überarbeitungen zum CSS Box-Modell (siehe Vorlesung 4). Seit April 2024 eine Recommendation.
- **CSS Color Module Level 4** [18]: Ermöglicht neue Arten, Farben in CSS zu spezifizieren. Mit April 2025 ein W3C Candidate Recommendation Draft.
- **CSS Backgrounds and Borders Module Level 3** [15]: Neue Optionen für Hintergründe und Rahmen (wie etwa abgerundete Ecken). Mit März 2024 ein Candidate Recommendation Draft.

- **CSS Images Module Level 3** [20]: Zuständig für die Einbindung von Bildern und Farbverläufen, im Dezember 2023 ein Candidate Recommendation Draft.
- **CSS Fonts Module Level 3** [19]: Ermöglicht es, herunterladbare Schriften zu verwenden. Seit 20. September 2018 eine Recommendation.
- **CSS Multi-column Layout Module** [21]: Mehrspaltiges Anordnen von Text. Seit Mai 2024 ein Candidate Recommendation Snapshot.
- **CSS Cascading and Inheritance Level 5** [17]: Ergänzt die ursprünglich in [11] definierte Kaskade um neue CSS-Funktionen. Der aktuelle Candidate Recommendation Snapshot stammt vom Jänner 2022.
- **CSS Text Module Level 3** [24]: Definiert erweiterte Eigenschaften zur Textmanipulation. Seit September 2024 ein Candidate Recommendation Draft.
- **CSS Transitions Level 1** [26]: Ermöglicht sanfte Übergänge bei Zustandsänderungen, etwa bei Mouse-Over-Effekten. Seit 2018 ein Working Draft.
- **CSS Animations Level 1** [14]: Keyframe-Animationen – im Jahr 2023 ein Working Draft.

Darüber hinaus gibt es noch eine Vielzahl weiterer Module, deren Aufzählung den Rahmen dieses Skriptums sprengen würde. Eine komplette Liste mit Status und Prioritäten kann unter <https://www.w3.org/Style/CSS/current-work> abgerufen werden.

3.2 Grundlegende Syntax

Eine *CSS-Regel* ist wie folgt aufgebaut. Optionale Angaben stehen in eckigen Klammern:

```
1 Selektor [, Selektor_2, ...] {  
2   Eigenschaft_1: Wert_1 [!important];  
3   Eigenschaft_2: Wert_2 [!important];  
4   ...  
5   Eigenschaft_n: Wert_n [!important];  
6 }
```

Am Anfang jeder Angabe steht ein *Selektor*. Er gibt an, worauf sich die Regel beziehen soll (z. B. auf ein `<h1>`-Element). Es ist möglich, noch weitere Selektoren mit Komma getrennt anzugeben.

Danach folgt eine öffnende geschweifte Klammer. Sie zeigt den Beginn der Liste an Eigenschaften an, also der Angaben, die aussagen, wie das selektierte Element aussehen soll. Diese Angaben bestehen aus dem Namen der CSS-Eigenschaft und einem dazugehörigen Wert, getrennt durch einen Doppelpunkt, abgeschlossen durch ein Semikolon. Wird zusätzlich noch das Schlüsselwort `!important` angegeben, erhält die Regel zusätzliche Gewichtung. Die schließende geschweifte Klammer beendet die CSS-Regel.

3.2.1 Beispiele

In den folgenden Beispielen wird die Schriftfarbe mit der Eigenschaft `color` verändert. `font-family` wird verwendet, um die Schriftart anzupassen.

Folgender Code setzt die Schriftfarbe aller `<h1>`-Überschriften auf grün.

```
1 h1 {  
2   color: green;  
3 }
```

Im folgenden Code werden Schriftart (Arial) und Schriftfarbe (rot) für alle `<h2>`-Überschriften mit einer Regel gesetzt. Die `!important`-Deklaration bei der zweiten Eigenschaft gibt dieser Angabe besonderes Gewicht.

```
1 h2 {  
2   color: #FF0000;  
3   font-family: Arial !important;  
4 }
```

Der folgende Code setzt die Schriftfarbe blau; sowohl für `<h3>`-Elemente als auch für alle Absätze (`<p>`).

```
1 h3, p {  
2   color: #0000FF;  
3 }
```

3.2.2 Kommentare

CSS-Kommentare werden mit einem Schrägstrich, gefolgt von einem Stern-Symbol geöffnet und in der umgekehrten Reihenfolge wieder geschlossen. Ein CSS-Kommentar sieht also wie folgt aus:

```
1 /* Dies ist ein CSS-Kommentar */
```

3.2.3 Gruppierung von Regeln

Wie im zweiten Beispiel ersichtlich, können zwei Angaben, die auf dasselbe Element wirken sollen, in einer Regel zusammengefasst werden. Dies ist aufgrund der Übersichtlichkeit und Lesbarkeit eine übliche Vorgehensweise. Es ist jedoch genauso möglich, zwei getrennte Regeln mit je einer Angabe zu schreiben.

Folgender Code ist also genauso möglich ...

```
1 h2 {  
2   color: #FF0000;  
3   font-family: Arial !important;  
4 }
```

...wie jener:

```
1 h2 {  
2   color: #FF0000;  
3 }  
4 h2 {  
5   font-family: Arial !important;  
6 }
```

Im dritten Beispiel wurde eine Eigenschaft zwei unterschiedlichen Elementen in einer Regel zugewiesen. Auch diese Regel kann auf zwei einzelne aufgeteilt werden, ohne das Ergebnis zu beeinflussen.

So ist also sowohl dies möglich ...

```
1 h3, p {  
2   color: #0000FF;  
3 }
```

...wie auch dies:

```
1 h3 {  
2   color: #0000FF;  
3 }  
4 p {  
5   color: #0000FF;  
6 }
```

3.3 Arten der Anwendung (Einbindung)

CSS-Regeln können auf vier Arten angewandt werden:

- als Attribut direkt beim Element (**style**-Attribut),
- direkt im **<head>**-Bereich des HTML-Dokuments (**<style>**-Element),
- verlinkt im **<head>**-Bereich des HTML-Dokuments (**<link>**-Element),
- eingebunden aus weiteren Stylesheets mittels **@import**.

3.3.1 Direkt beim Element

Es besteht die Möglichkeit, CSS-Eigenschaften direkt im HTML-Code zu spezifizieren. Möglich ist dies mit dem **style**-Attribut, welches zu den Universalattributen zählt und somit mit jedem HTML-Element verwendet werden kann. Die Eigenschaften werden als Wert des **style**-Attributs angegeben. Jedes Eigenschaft-Wert-Paar sollte mit Semikolon abgeschlossen werden, um Probleme zu vermeiden.

Um die Farbe einer **<h1>**-Überschrift auf grün zu setzen, kann das folgende Markup verwendet werden.

```
1 <h1 style="color: green;">
```

Auch wenn es sich dabei um eine erlaubte und auch valide Methode handelt, so sollte sie doch sehr sparsam eingesetzt werden. Denn die Angabe von Formatierungen direkt im HTML-Markup widerspricht dem rein bedeutungsgebenden Charakter von HTML. Es ist zudem nur möglich, Eigenschaften einem bestimmten HTML-Element zuzuweisen. Generelle Zuweisungen (z. B. Formatierungen aller **<h1>**-Elemente) sind nicht möglich. Daher ergibt sich auch hier eine sehr starke Änderungsfeindlichkeit.

3.3.2 Direkt im Kopfbereich

Um Formatierungen für eine gesamte HTML-Datei anzugeben, können diese direkt in der jeweiligen Datei notiert werden. Dazu muss im **<head>**-Bereich das Element **<style>** platziert werden (Abschluss mit **</style>**). Innerhalb dieses Elements können beliebig viele CSS-Regeln platziert werden. Früher wurde beim **<style>**-Element das Attribut **type** mit dem Wert **text/css** angegeben, um zu spezifizieren, dass es sich um CSS-Angaben handelt. Seit HTML5 ist dies der Standardwert, daher kann (und sollte) das Attribut weggelassen werden.

Folgender Code setzt die Farbe grün für alle **<h1>**-Überschriften im Dokument.

```
1 <head>  
2   ...  
3   <style>  
4     h1 {
```

```
5     color: green;
6   }
7   </style>
8 </head>
```

Auch diese Variante sollte kritisch betrachtet werden. Während sie zwar mehr Flexibilität bietet, als die direkte Angabe beim Element, so bedeutet sie doch einen großen Eingriff in die HTML-Datei. Diese wird durch die Style-Regeln zusätzlich vergrößert und muss jedes Mal geändert werden, wenn Regeln verändert werden. Diese Variante der Einbindung eignet sich meist dann, wenn nur eine einzelne HTML-Datei vorliegt und auch die Anzahl der CSS-Regeln überschaubar ist.

3.3.3 Als eigenes Dokument

CSS-Angaben werden am Besten in eine eigene Datei ausgelagert. Diese Datei hat die Endung `.css` und ist eine reine Textdatei. Sie enthält nur CSS-Regeln, kein HTML-Markup (also kein `<style>`-Element oder andere Elemente).

Die Einbindung dieser Datei erfolgt wiederum im `<head>` des HTML-Dokuments über das `<link>`-Element. Dieses wird generell dazu verwendet, um externe Ressourcen in das aktuelle Dokument einzubinden. Das Element benötigt zwingend die Attribute `rel` und `href`. `rel` spezifiziert die Beziehung, in der das verlinkte Dokument zum aktuellen steht. Im Falle einer CSS-Datei muss der Wert `stylesheet` lauten. Das `href`-Attribut gibt die Quelle des einzubindenden Dokuments an und sollte idealerweise ein relativer Link auf die CSS-Datei sein. Die Angabe des `type`-Attributs (mit dem Wert `text/css`) kann wie bei der direkten Einbindung entfallen.

Das HTML-Markup zur Einbindung einer externen CSS-Datei sieht wie folgt aus:

```
1 <head>
2   ...
3   <link rel="stylesheet" href="styles.css">
4 </head>
```

Diese Methode ist die flexibelste und sollte, wann immer möglich, verwendet werden. Es besteht lediglich eine (notwendige) Verbindung zwischen HTML-Dokument und CSS-Datei, die nur dann angepasst werden muss, falls sich der Name der CSS-Datei ändert oder eine andere eingebunden werden soll. Ansonsten muss das HTML-Dokument nicht verändert werden, wenn neue Formatierungen hinzugefügt oder bestehende geändert werden sollen.

3.3.4 Einbinden aus weiteren Stylesheets

Mit einer so genannten `@import`-Regel lassen sich weitere, externe Stylesheets mit einer Angabe importieren. Nach dem Schlüsselwort folgt die Spezifizierung des Stylesheets mittels `url`-Angabe. Die Anweisung könnte direkt im `<style>`-Element verwendet werden um dort ein Stylesheet zu laden:

```
1 <head>
2   ...
3   <style>
4     @import url("style.css");
5   </style>
6 </head>
```

Die weitaus gebräuchlichere Variante ist jedoch, in einem externen Stylesheet weitere Stylesheets zu importieren. So muss mit über `<link>` nur ein zentrales Stylesheet inkludiert werden, welches wiederum alle weiteren mittels `@import` lädt:

```
1 /* main.css */
2 @import url("base.css");
3 @import url("layout.css");
4 @import url("navigation.css");
```

Sollen die Angaben eines Stylesheets gleich einem Kaskaden-Layer zugewiesen werden, kann dieser mit angegeben werden:

```
1 @import url("layout.css") layer(components);
```

Generell ist bei `@import` anzumerken, dass jede `@import`-Angabe einen zusätzlichen Request erzeugt und es in der Praxis daher eher sparsam eingesetzt werden sollte.

3.4 Arten von Selektoren

Der Selektor einer CSS-Regel bestimmt, worauf genau die Eigenschaften der Regel angewandt werden. Dabei gibt es eine Reihe an unterschiedlichen Möglichkeiten, welche im Nachfolgenden erläutert werden.

3.4.1 Element-/Typselektoren

Elementselektoren, auch *Typselektoren* genannt, formatieren immer ein ganz bestimmtes HTML-Element (z. B. `<body>`, `<h1>`, `<em>`, `<span>`, `<div>` etc.). Dabei werden alle Vorkommen des Elements auf der Seite formatiert.

Folgendes Beispieldokument soll formatiert werden:

```
1 <body>
2   <h1>Dies ist die Überschrift</h1>
3   <p>Dies ist ein Absatz und <em>das hier</em> ist hervorgehoben.</p>
4 </body>
```

Folgender CSS-Code soll zur Anwendung kommen. Zusätzlich zu den bereits erwähnten Eigenschaften `color` und `font-family` kommen die Angaben `background-color` zur Angabe der Hintergrundfarbe sowie `font-size` zur Angabe der Schriftgröße zum Einsatz. Das Ergebnis ist in Abbildung 3.1 ersichtlich.

```
1 body {
2   background-color: #00FFFF;
3 }
4 h1 {
5   color: #0000FF;
6 }
7 p {
8   color: #FFFFFF;
9   background-color: #808080;
10 }
11 em {
12   font-family: "Courier New", Courier, monospace;
13   font-size: 24px;
14 }
```




Abbildung 3.1: Alle Formatierungen sind mit Typselektoren durchgeführt. Das gesamte Dokument (`<body>`) erhält eine Hintergrundfarbe, Überschriften (`<h1>`) werden blau eingefärbt, Absätze (`<p>`) erhalten eine graue Hintergrundfarbe und weiße Schrift und betonte Stellen (`<em>`) bekommen eine andere Schriftart und -größe zugewiesen.

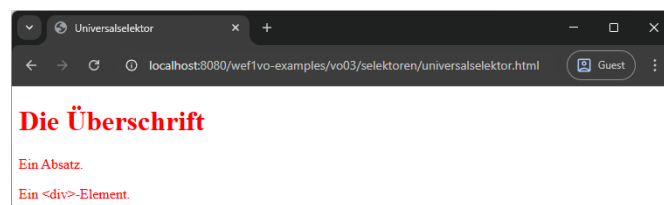


Abbildung 3.2: Der Universalselektor wählt jedes Element im Dokument aus. Daher sind alle rot dargestellt.

3.4.2 Universalselektor

Der *Universalselektor* funktioniert wie ein Typselektor, nur werden alle Elemente eines HTML-Dokuments ausgewählt. Er wird durch das Stern-Symbol (*) repräsentiert. In Kombination mit einem Klassen-, ID- oder Attributselektor kann der Stern auch entfallen, d. h. `*.crazy` entspricht `.crazy` (Details siehe folgende Abschnitte).

Folgendes HTML-Markup enthält drei unterschiedliche Elemente (eine Überschrift, einen Absatz und einen generischen Abschnitt).

```
1 <body>
2   <h1>Die Überschrift</h1>
3   <p>Ein Absatz.</p>
4   <div>Ein &lt;div>-Element.</div>
5 </body>
```

Mit Hilfe des Universalselektors im folgenden CSS-Code wird dieselbe Formatierung für alle Elemente im Dokument, also `<h1>`, `<p>`, `<div>` und auch `<body>` angewandt. Das Ergebnis ist in Abbildung 3.2 ersichtlich.

```
1 * {
2   color: #FF0000;
3 }
```

3.4.3 Klassenselektoren

Klassenselektoren ermöglichen eine Definition von Formatierungen unabhängig vom Element. Sie beginnen in CSS mit einem Punkt (z. B. `.crazy`) und wirken auf alle Elemente, die ein Attribut `class` besitzen, dessen Wert der Name des Klassenselektors ist (z. B.



Abbildung 3.3: Die Klasse `.big` wird sowohl auf die `<h1>`-Überschrift als auch auf den mit `<span>` markierten Bereich angewandt. Die Klasse `.crazy` ist der Anwendung auf Absätze (`<p>`) vorbehalten, da sie in CSS als `p.crazy` angegeben wurde.

`class="crazy"`). Klassen mit demselben Namen dürfen im HTML-Markup bei beliebig vielen Elementen vorkommen, somit sind Klassenselektoren universell einsetzbar. Soll die Verwendung dennoch auf einen bestimmten Elementtyp eingeschränkt werden, kann dieser Typ dem Klassenselektor vorangestellt werden (z. B. `p.crazy`).

Folgendes Markup demonstriert den Einsatz von Klassenselektoren. Alle Elemente mit einem `class`-Attribut werden formatiert:

```
1 <body>
2   <p class="big">Absatz 1 schaut so aus.</p>
3   <p class="crazy">Absatz 2 etwas anders.</p>
4   <p>Aber die Stelle <span class="big">hier</span> wieder wie bei 1.</p>
5 </body>
```

Der nachfolgende CSS-Code kommt dabei zum Einsatz. Das Ergebnis ist in Abbildung 3.3 ersichtlich.

```
1 .big {
2   font-size: xx-large;
3   font-family: Arial, Helvetica, sans-serif;
4 }
5 p.crazy {
6   color: #FF00FF;
7   background-color: #00FF00;
8 }
```

3.4.4 ID-Selektoren

ID-Selektoren arbeiten ähnlich wie Klassenselektoren. Auch sie dienen zur vom Element losgelösten Formatierung. Anstatt mit einem Punkt beginnen sie in CSS mit einem Raute-Zeichen (z. B. `#primary`) und wirken auf das Element mit dem Attribut `id`, dessen Wert mit dem Namen des ID-Selektors übereinstimmt (z. B. `id="primary"`). Hier offenbart sich auch der zentrale Unterschied: `id` ist zwar wie `class` ein Universalattribut, darf also bei jedem Element verwendet werden, der Wert eines jeden `id`-Attributs muss aber einzigartig sein. Somit kann ein ID-Selektor auch nur auf ein einzelnes Element im Markup angewandt werden. Es kann wie bei Klassen eine Einschränkung auf einen Elementtyp vorgenommen werden (z. B. `p#primary`), diese ist jedoch nicht wirklich nötig.

Folgendes HTML-Markup enthält eine ID, welche formatiert werden soll.



Abbildung 3.4: Der zweite Absatz besitzt das `id`-Attribut, welches über CSS formatiert wird. Es darf kein zweites Element mit derselben ID geben.

```

1 <body>
2   <p>Das ist der erste Absatz.</p>
3   <p id="primary">Aber der zweite ist viel wichtiger.</p>
4   <p>Der dritte wieder nicht.</p>
5 </body>

```

Die ID ist in folgendem CSS-Code definiert. Abbildung 3.4 zeigt das Ergebnis.

```

1 #primary {
2   font-size: xx-large;
3   color: red;
4 }

```

3.4.5 Attributselektoren

Attributselektoren erlauben die Formatierung von Elementen auf Basis von Attributen (und deren Werten) im HTML-Markup. Attributselektoren werden in eckigen Klammern angegeben, vor den Klammern steht meist noch ein Element, sodass nur Attributangaben bei einem bestimmten Typ zum Zug kommen. Die folgenden Selektoren stehen zur Verfügung:

- `element[att]`: Selektiert alle Elemente, bei denen das Attribut `att` gesetzt ist (egal, welchen Wert es hat). Zum Beispiel: `p[title]` selektiert alle Absätze, bei denen das `title`-Attribut gesetzt ist.
- `element[att="wert"]`: Selektiert alle Elemente, die ein Attribut `att` besitzen, dessen Wert genau `wert` lautet. Zum Beispiel: Der Selektor `input[type="text"]` liefert alle Eingabefelder, deren `type`-Attribut den Wert `text` hat.
- `element[att~="wert"]`: Wählt alle Elemente aus, die ein Attribut besitzen, das eine durch Leerzeichen getrennte Liste von Werten enthält, von denen einer genau `wert` entspricht. `p[class~="crazy"]` liefert nur Absätze, bei denen mehrere Klassen angegeben sind und eine davon `crazy` lautet, also z.B. Folgenden: `<p class="big crazy primary">`.
- `element[att|="wert"]`: Selektiert Elemente, die ein Attribut besitzen, dessen Wert mit `wert` beginnt und von einem Bindestrich gefolgt wird. Dies ist vor allem bei Sprachattributen interessant, z.B. selektiert `p[lang|="en"]` alle Absätze, deren `lang`-Attributwert mit `en-` beginnt, also etwa `<p lang="en-US">` oder `<p lang="en-UK">`.

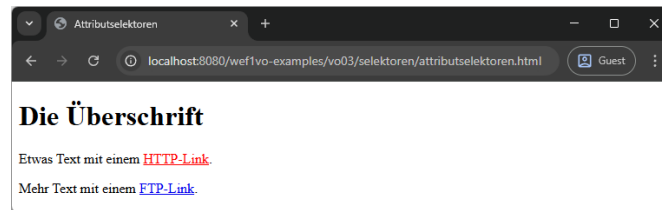


Abbildung 3.5: Der erste Link beginnt mit `http://` und wird daher rot dargestellt. Der zweite ist ein FTP-Link und wird laut Standardformatierung aus dem Browser-Stylesheet blau dargestellt.

- `element[att^="wert"]`: Liefert alle Elemente, deren Attribut `att` mit dem Wert `wert` beginnt. So liefert `a[href^="http://"]` all jene absoluten Links, die mit `http://` beginnen (also keine FTP-, HTTPS- oder andere Links).
- `element[att$="wert"]`: Selektiert alle Elemente, die ein Attribut `att` besitzen, dessen Wert mit `wert` endet. `a[href$=".html"]` erlaubt es, jene Links zu formatieren, die explizit auf HTML-Dateien verweisen (und nicht auf PHP, DOCX, MP3 oder Ähnliches).
- `element[att*="wert"]`: Gibt Elemente zurück, in deren Attribut `att` `wert` enthalten ist. Dies könnte verwendet werden um Links zu selektieren, in denen ein Benutzer*innenname vorkommt. So kann mit `a[href*="TheHoff"]` das Element `<a href="https://www.last.fm/user/TheHoff/charts">` selektiert werden.

Folgendes HTML-Markup beinhaltet zwei Links mit unterschiedlichen Protokollen (HTTP und FTP).

```
1 <body>
2   <h1>Die Überschrift</h1>
3   <p>Etwas Text mit einem <a href="http://speedtest.tele2.net/">HTTP-Link</a>.</p>
4   <p>Mehr Text mit einem <a href="ftp://speedtest.tele2.net/">FTP-Link</a>.</p>
5 </body>
```

Im CSS-Code werden alle Links formatiert, die mit `http://` beginnen. Das Ergebnis ist in Abbildung 3.5 dargestellt.

```
1 a[href^="http://"] {
2   color: #FF0000;
3 }
```

Die Groß- und Kleinschreibung beim Vergleich von Attributen und Attributwerten macht in den meisten Fällen einen Unterschied. Der Selektor `a[href^="http://"]` ergibt *keine* Übereinstimmung, wenn das Attribut im HTML-Markup `HREF` geschrieben ist oder der Wert mit `HTTP://` beginnt. Möchte man unabhängig von Groß- und Kleinschreibung selektieren (case-insensitive), kann ein `i` mit Leerzeichen getrennt in jeden Selektor gesetzt werden. Umgekehrt kann auch Case-Sensitivity erzwungen werden, indem ein `s` mit Leerzeichen getrennt in den Selektor platziert wird:

```
1 a[href^="http://" i] { ... } /* case-insensitive */
2 a[href^="http://" s] { ... } /* case-sensitive */
```

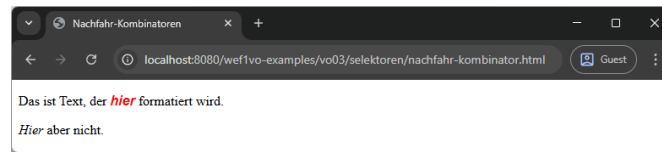


Abbildung 3.6: Die Regel selektiert nur jenes `<em>`-Element, das innerhalb des `<div>`-Elements liegt. Auch wenn dort noch `<p>` und `<strong>` in der Hierarchie dazwischen liegen, greift der Nachfahr-Kombinator.

3.4.6 Nachfahr-Kombinatoren

Nachfahr-Kombinatoren (engl. „descendant combinators“) erlauben es, Elemente zur Formatierung zu selektieren, wenn sie ein Nachfahre eines anderen Elements sind. Von einem Nachfahren wird gesprochen, wenn das Element in einem anderen verschachtelt bzw. enthalten ist – die Verschachtelungstiefe ist dabei egal. Damit ist es etwa möglich, nur all jene `<em>`-Elemente zu selektieren (und somit zu formatieren), die innerhalb eines `<div>`-Elements liegen. `<em>`-Elemente in Absätzen (`<p>`) werden nicht formatiert. Ob aber nun das `<em>`-Element direkt innerhalb von `<div>`-liegt, oder das `<em>`-Element von einem `<strong>`-Element und dieses dann im `<div>`-Element liegt, spielt keine Rolle. `<em>` ist zwar tiefer verschachtelt, aber die Verschachtelungstiefe ist, wie schon erwähnt, nebensächlich.

Um einen derartigen Nachfahr-Kombinator zu erstellen, werden die ineinander verschachtelten Elemente mit Leerzeichen getrennt angegeben, dabei besteht das beinhaltende Element immer links vom enthaltenen. Um also alle `<em>`-Elemente zu selektieren, die in einem `<div>`-Element enthalten sind, muss der Kombinator `div em` lauten. Als Eselsbrücke kann ein Lesen von rechts nach links dienen: „Selektiere alle `em`-Elemente, die in einem `div` liegen“.

Auf diese Art und Weise können beliebig viele Hierarchien erstellt werden. Selbstverständlich können auch andere Selektortypen verwendet werden: `#content .crazy p` selektiert etwa alle Absätze (`<p>`), die in einem Element mit der Klasse `crazy` liegen, welches wiederum in einem Element mit der ID `content` enthalten ist.

Ein Beispiel – folgendes HTML-Markup soll formatiert werden:

```
1 <body>
2   <div>
3     <p>Das ist Text, der <strong><em>hier</em></strong> formatiert wird.</p>
4   </div>
5   <p><em>Hier</em> aber nicht.</p>
6 </body>
```

Die folgende CSS-Regel kommt zur Anwendung, das Ergebnis ist in Abbildung 3.6 dargestellt.

```
1 div em {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

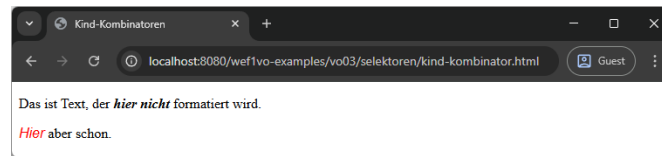


Abbildung 3.7: Die CSS-Regel hat nur Auswirkungen auf den zweiten Absatz, da hier das `<em>`-Element ein Kindelement des Absatzes ist. Im ersten Absatz liegt das `<strong>`-Element in der Hierarchie noch dazwischen.

3.4.7 Kind-Kombinatoren

Kind-Kombinatoren (engl. „child combinators“) funktionieren ähnlich wie Nachfahr-Kombinatoren, selektieren jedoch nur jene Elemente, die genau eine Ebene unterhalb eines anderen liegen, also Kinder (nicht einfach nur Nachfahren) sind. Der Kombinator besteht aus zwei oder mehreren Elementen, die durch das „größer“ Zeichen (`>`) getrennt sind. `p > em` selektiert also nur jene `<em>` Elemente, die direkt innerhalb eines Absatzes liegen.

Es wird dasselbe Markup wie zuvor (mit leicht verändertem Text) verwendet.

```
1 <body>
2   <div>
3     <p>Das ist Text, der <strong><em>hier nicht</em></strong> formatiert wird.</p>
4   </div>
5   <p><em>Hier</em> aber schon.</p>
6 </body>
```

Nun kommt folgende CSS-Regel zur Anwendung, das Ergebnis ist in Abbildung 3.7 dargestellt.

```
1 p > em {
2   color: #FF0000;
3   font-family: Arial, Helvetica, sans-serif;
4 }
```

3.4.8 Nächste Geschwister-Kombinatoren

Nächste Geschwister-Kombinatoren (engl. „next-sibling combinators“) sind die dritte Art und Weise, Formatierungen im Bezug auf die HTML-Struktur vorzunehmen. Hiermit können Elemente selektiert werden, die direkt nach einem anderen auf derselben Ebene vorkommen, also Geschwister sind. Ein Beispiel wäre, alle Absätze zu selektieren, die direkt nach einer `<h1>`-Überschrift kommen.

Der Kombinator besteht aus zwei oder mehr Elementen, die durch das Plus-Zeichen (`+`) getrennt sind. `h1 + p` selektiert eben genau jenen Absatz, der direkt nach der Überschrift platziert ist.

Auch das folgende HTML-Markup stellt dieses Beispiel dar.

```
1 <body>
2   <h1>Hier die Überschrift</h1>
3   <p>Hier der erste, formatierte Absatz.</p>
4   <p>Hier der zweite, unformatierte.</p>
5 </body>
```

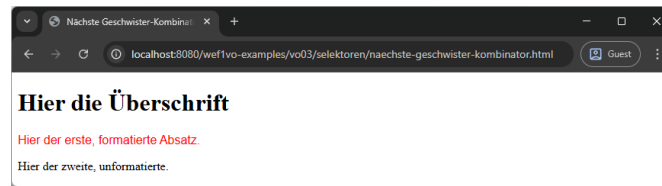


Abbildung 3.8: Nur der erste Absatz wird formatiert. Obwohl beide Absätze *Geschwister* der Überschrift sind (da sie nicht verschachtelt sind sondern auf derselben Ebene liegen), wird nur der direkt auf die Überschrift folgende berücksichtigt.

Die CSS-Regel formatiert nun den ersten Absatz, das Ergebnis ist in Abbildung 3.8 dargestellt.

```
1 h1 + p {  
2   color: #FF0000;  
3   font-family: Arial, Helvetica, sans-serif;  
4 }
```

3.4.9 Nachfolgende Geschwister-Kombinatoren

Der *nachfolgende Geschwister-Kombinator* (engl. „subsequent-sibling combinator“) selektiert alle Elemente, die auf ein anderes, aber nicht unbedingt direkt danach folgen. Der Kombinator besteht aus zwei oder mehreren Elementen, die durch das Tilde-Zeichen (~) getrennt sind. `div ~ p` selektiert alle Absätze, die nach einem `<div>`-Element kommen und dasselbe Elternelement haben.

Das folgende Markup enthält drei Absätze, einer innerhalb eines `<div>`-Elements (Kindelement), zwei andere danach (Geschwisterelemente).

```
1 <body>  
2   <div>  
3     <p>Erster Absatz.</p>  
4   </div>  
5   <p>Zweiter Absatz.</p>  
6   <p>Dritter Absatz.</p>  
7 </body>
```

Die CSS-Regel formatiert nur den zweiten und dritten Absatz, das Ergebnis ist in Abbildung 3.9 dargestellt.

```
1 div ~ p {  
2   color: #FF0000;  
3   font-family: Arial, Helvetica, sans-serif;  
4 }
```

3.4.10 Pseudoelemente

Bis jetzt wurden alle Formatierungen auf Elemente, Klassen, oder IDs angewandt – also auf Dinge, die im HTML-Markup ersichtlich waren (entweder als Element oder als Attribut). Es gibt jedoch auch Szenarien, in denen Eigenschaften formatiert werden sollen, die nicht im Markup so einfach repräsentiert sind. Zu diesem Zweck wurde das

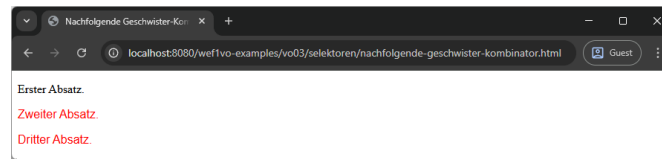


Abbildung 3.9: Alle Absätze, die Geschwister des `<div>`-Elements sind (also auf derselben Ebene liegen), werden formatiert.

Konzept von Pseudoelementen und auch der im nächsten Abschnitt beschriebenen Pseudoklassen eingeführt.

Pseudoelemente sind eine Abstraktion, auf Teile eines bestehenden Elements zu verweisen, für die es keine Repräsentation gibt (z. B. die erste Zeile eines Absatzes). Ein Pseudoelement-Selektor besteht aus dem Element, gefolgt von zwei Doppelpunkten und dem eigentlichen Pseudoelement, also z. B. `p::first-line` um die erste Zeile eines Absatzes zu selektieren.

Eine Auswahl der wichtigsten Pseudoelemente:

- `::first-line`: Selektiert die erste Zeile eines Blockelements (ein Element, das eine neue Zeile beginnt, wie etwa ein Absatz), entweder definiert durch einen automatischen Umbruch oder durch ein `<br>`-Element (z. B. `p::first-line`).
- `::first-letter`: Selektiert den ersten Buchstaben eines beliebigen Blockelements (z. B. `p::first-letter`).
- `::selection`: Erlaubt es, Styles auf den (etwa mit der Maus) selektierten Text auszuführen.
- `::before`: Ermöglicht es, vor dem selektierten Element Text einzufügen (z. B. `p::before {content: "Text:"}` fügt vor allen Absätzen auf der Seite das Wort „Text:“ ein). Da sie Inhalt *erzeugen*, brauchen Sie die Eigenschaft `content`.
- `::after`: Fügt Text nach dem selektierten Element ein.
- `::marker`: Erlaubt es, Listenzeichen von `<li>`-Elementen zu formatieren. Dieses kann mit `content` gesetzt werden.
- `::placeholder`: Formatiert die Platzhalter-Angabe in einem Eingabfeld.

Das HTML-Markup besteht nur aus einem Absatz:

```
1 <body>
2   <p>Dieser Absatz hat so viel Text, dass er bei schmalem Browserfenster umbricht
   und somit weitere Zeilen entstehen. Beim Markieren wird der Bereich gelb
   hinterlegt.</p>
3 </body>
```

Der CSS-Code enthält drei der erwähnten Pseudoelemente, die auf Absätze wirken. Das Ergebnis ist in Abbildung 3.10 dargestellt.

```
1 p::first-line {
2   color: fuchsia;
3 }
4 p::first-letter {
5   font-size: xx-large;
6 }
7 p::selection {
```

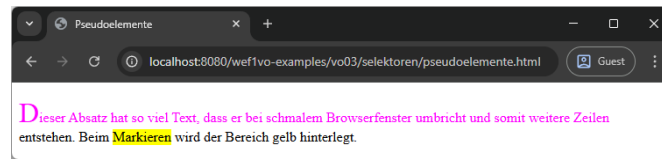



Abbildung 3.10: Die ersten Zeile des Absatzes bekommen über das Pseudoelement die Farbe „Fuchsia“, der erste Buchstabe wird größer dargestellt. Wird ein Bereich selektiert, wird er gelb hinterlegt.

```
8 background-color: yellow;
9 }
```

3.4.11 Pseudoklassen

Pseudoklassen erlauben es, ein Element mit bestimmten Eigenschaften bzw. in einem bestimmten Zustand zu selektieren. Der Selektor besteht aus dem Element, einem Doppelpunkt und der Pseudoklasse, also z. B. `a:link` für einen unbesuchten oder `a:visited` für einen besuchten Link.

Einige wichtige Pseudoklassen:

- `:first-child`: Selektiert jenes Element, welches das erste Kindelement eines anderen Elements ist. Enthält ein `<div>`-Element etwa drei Absätze (`<p>`), selektiert `div p:first-child` nur den ersten der drei.
- `:link`: Selektiert nur jene Links, die noch nicht besucht wurden (z. B. `a:link`).
- `:visited`: Selektiert nur jene Links, die bereits besucht wurden (z. B. `a:visited`).
- `:hover`: Selektiert jenen Link, über den Benutzer*innen gerade die Maus bewegen (z. B. `a:hover`).
- `:active`: Selektiert jenen Link, der gerade angeklickt wird (zwischen Mausklick und Loslassen der Taste – `a:active`).
- `:focus`: Selektiert jenes Element, das zur Zeit den Fokus hat (`a:focus`, wenn der Link z. B. mit Tab angewählt wurde).
- `:nth-child()`, `:nth-last-child()`: Selektiert das n -te Element basierend auf ihrer Position. Erlaubt als Argument einfache Skalare (z. B. `li:nth-child(3)`) oder eine Angabe im Format $a \cdot n + b$. a und b sind beliebige skalare Werte, n entspricht einer Zählervariable (wie etwa i in for-Schleifen), die von 0 weg hochgezählt wird. `li:nth-child(3n+5)` selektiert also das 5., 8., 11. usw. `<li>`-Element. `:nth-last-child()` beginnt von hinten zu zählen.
- `:nth-of-type()`, `:nth-last-of-type()`: Hat dieselbe Syntax wie `:nth-child()`, selektiert jedoch das n -te Element einer Art. `section p:nth-child(3)` wählt etwa den dritten Absatz innerhalb eines Abschnitts aus, selbst wenn darin auch `<div>`-Elemente vorkommen. `:nth-last-of-type()` zählt von hinten.
- `:only-child`: Selektiert Elemente, die keine Geschwister haben.
- `:only-of-type`: Selektiert Elemente, die keine Geschwister desselben Typs haben.
- `:empty`: Selektiert leere Elemente.
- `:root`: Wählt das Wurzelement – in HTML immer `<html>` – aus.

- **:target**: Formatiert ein Verweisziel. Wird etwa ein `<section>` Element mit einem internen Link angesprungen, kann mit `section:target` dieser Abschnitt z.B. visuell hervorgehoben werden.
- **:enabled**, **:disabled**, **:checked**: Damit können Formularelemente ausgewählt werden, die aktiv, inaktiv oder ausgewählt sind (letzteres nur für Checkboxes und Radio Buttons).

In folgendem HTML-Snippet kommen ein Link in einem Absatz, eine Liste und ein Abschnitt mit Absätzen und einem allgemeinen Bereich vor.

```

1 <body>
2   <p>Die <a href="https://www.fh-ooe.at/">FH OÖ</a> im Web.</p>
3   <ol>
4     <li>Ein Listenelement.</li>
5     <li>Noch eins.</li>
6     <li>Und noch eins.</li>
7     <li>Und ein viertes.</li>
8   </ol>
9   <section>
10    <p>Ein Absatz.</p>
11    <div>Ein generischer Bereich.</div>
12    <p>Noch ein Absatz.</p>
13  </section>
14 </body>

```

Der CSS-Code enthält eine Pseudoklasse, die den Link einfärbt, eine weitere selektiert alle geraden Listenelemente und die dritte formatiert lediglich den zweiten Absatz innerhalb eines Abschnitts. Das Ergebnis ist in Abbildung 3.11 dargestellt.

```

1 a:link {
2   color: #FF0000;
3   font-size: large;
4 }
5 li:nth-child(2n) {
6   color: green;
7 }
8 section p:nth-of-type(2) {
9   background-color: lightgray;
10 }

```

Funktionale Pseudoklassen

Neben den ursprünglichen Pseudoklassen, die Zustände eines Elements beschreiben, existieren auch sogenannte *funktionale Pseudoklassen*. Sie enthalten andere Selektoren in Klammern und dienen dazu, komplexe Regeln zu vereinfachen oder zu kombinieren.

:is(): Erlaubt es, mehrere Selektoren zusammenzufassen. Jedes Element, das durch einen der darin enthalten Selektoren angesprochen wird, wird auch formatiert (in diesem Fall Überschriften erster und zweiter Ordnung). Diese Pseudoklasse ist vor allem bei komplizierteren Selektoren (meist Kombinatoren) nützlich.

```

1 :is(h1, h2) {
2   color: purple;
3 }

```

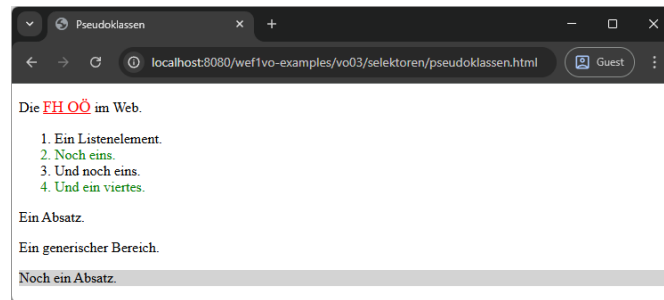


Abbildung 3.11: Die Pseudoklasse `a:link` färbt den Link nur dann rot, wenn er noch nicht besucht wurde, jedes zweite (gerade) Listenelement wird grün dargestellt und nur der zweite Absatz in einem Abschnitt erhält einen hellgrauen Hintergrund.

Neben `:is()` existiert auch noch `:where()`, das quasi identisch funktioniert. Unterschiede existieren nur in der Berechnung der Specificity. Bei `:is()` erhält der Selektor den Wert des spezifischsten Elements, bei `:where()` ist die Specificity immer 0. Mehr dazu in Abschnitt 3.6.6. Hier werden alle Überschriften von der ersten bis zur dritten Ordnung selektiert:

`:not()`: Schließt Elemente aus, die einem bestimmten Selektor entsprechen. So selektiert etwa die folgende Angabe alle Absätze *bis* auf den ersten.

```
1 p:not(:first-child) {
2   color: blue;
3 }
```

`:has()`: Diese Pseudoklasse wählt Elemente aus, die bestimmte Nachfahren enthalten oder Geschwister haben. Im folgenden Beispiel werden etwa nur Artikel formatiert, die auch einen Absatz enthalten:

```
1 article:has(p) {
2   background-color: lightgray;
3 }
```

Abbildung 3.12 zeigt ein Beispiel mit folgendem Markup, in dem die obigen Selektoren zur Anwendung kommen.

```
1 <h1>Überschrift</h1>
2
3 <article>
4   <h2>Artikelüberschrift</h2>
5 </article>
6
7 <article>
8   <h2>Artikelüberschrift</h2>
9   <section>
10    <p>Etwas Text davor</p>
11    <h3>Abschnittsüberschrift</h3>
12    <p>Etwas Text danach</p>
13  </section>
14 </article>
```

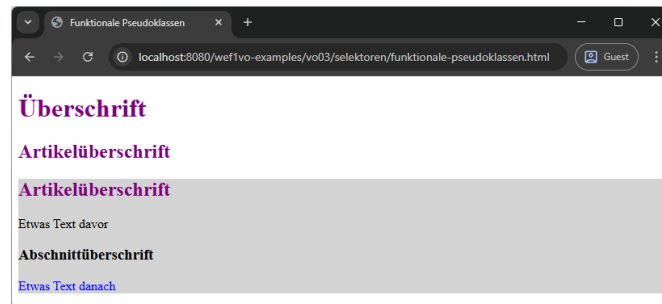


Abbildung 3.12: Alle `<h1>`- und `<h2>`-Überschriften sind violett, nur der Artikel mit den Absätzen ist hellgrau hinterlegt und der erste Absatz im Artikel wird *nicht* formatiert.

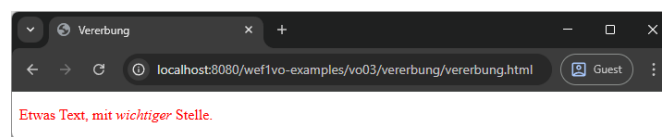


Abbildung 3.13: Vererbung bewirkt, dass auch der Inhalt des `<em>`-Elements rot eingefärbt wird.

Unter <https://flukeout.github.io/> können die verschiedenen CSS-Selektoren spielerisch erlernt und ausprobiert werden.

3.5 Vererbung von Eigenschaften

Selektoren bestimmen, auf welches Element eine Formatierung angewandt wird. Doch oft ist es nicht nur dieses Element, sondern auch alle Nachfahren (d. h. im Element verschachtelten Elemente), die diese Formatierung erhalten. Die Ursache ist der *Vererbungsmechanismus* (engl. „inheritance“), der in CSS zum Einsatz kommt.

Dieses Verhalten wird in der Regel kaum bewusst wahrgenommen, da es sehr natürlich implementiert ist. Gegeben sei folgendes Beispiel:

```
1 <p>Etwas Text, mit <em>wichtiger</em> Stelle.</p>
```

Nun soll mit der folgenden Regel der Absatz rot eingefärbt werden:

```
1 p {
2   color: #FF0000;
3 }
```

Die Vererbung bewirkt, dass der gesamte Absatz rot dargestellt wird (Abbildung 3.13). Gäbe es diesen Mechanismus nicht, wäre der betonte Teil weiterhin schwarz dargestellt und müsste separat formatiert werden. Soll der Inhalt von `<em>` tatsächlich schwarz dargestellt werden, müsste er noch extra formatiert werden.

Ob eine Eigenschaft vererbt wird, ist in der CSS-Spezifikation enthalten. Der Großteil aller Eigenschaften, wie Schriftart und -größe, Textfarben und -betonungen etc. wird vererbt.

Nicht vererbt werden Hintergründe (Hintergrundfarben und -bilder) sowie Rahmen und Abstände. Im Beispiel von Rahmen wäre dies absolut nicht wünschenswert. Nimmt

man das Beispiel von oben und setzt um den Absatz einen Rahmen, so würde das Wort im `<em>`-Element bei Vererbung einen eigenen kleinen Rahmen bekommen. Da dies in den seltensten Fällen gewünscht ist, ist hier die Vererbung außer Kraft gesetzt.

Soll eine Vererbung trotzdem erzwungen werden, kann dies mit dem Schlüsselwort **inherit** geschehen. Es greift auf die Werte von übergeordneten Elementen zu und verwendet diese für das aktuelle Element.

Folgender CSS-Code würde bewirken, dass alle Elemente auf der Seite rote Schrift und gelben Hintergrund verwenden. Die Farben sind für `<body>` definiert, alle anderen Elemente werden angewiesen, diese Werte zu übernehmen. Die **!important** Deklaration verleiht den Anweisungen zusätzliche Wichtigkeit und verhindert, dass sie gegebenenfalls überschrieben werden.

```
1 body {
2   color: red !important;
3   background-color: yellow !important;
4 }
5 * {
6   color: inherit !important;
7   background-color: inherit !important;
8 }
```

Um eine Eigenschaft wieder auf den Standardwert zu setzen, kann das Schlüsselwort **initial** verwendet werden. Mit **unset** wird eine Eigenschaft zurückgesetzt. Dies entspricht **inherit**, wenn sie vererbt war, sonst **initial**. **revert** setzt eine Eigenschaft ebenfalls zurück: auf **inherit** wenn sie vererbt war, oder sonst auf den Wert aus dem Browser-Stylesheet.

3.6 Die Kaskade

Das „C“ in CSS steht für „Cascading“, zu deutsch „kaskadierend“ (oder kaskadenförmig) und weist auf den Mechanismus hin, der bei der Bestimmung der zutreffenden Formatierung zur Anwendung kommt. Bevor ein Element sein Aussehen bekommt, wird mit diesem Regelwerk (definiert in [17, Abschnitt 6]) bestimmt, welches Aussehen dies ist, denn oftmals gibt es mehrere Definitionen, die zum Zug kommen könnten.

3.6.1 Filterung

Bevor die eigentliche Kaskade abgearbeitet wird, findet eine **Filterung** statt [17, Abschnitt 5]). D. h., es werden alle Deklarationen (CSS-Regeln) für das zu formatierende Element identifiziert. Dazu werden alle eingebundenen Stylesheets nach gültigen und für dieses Element zutreffenden Deklarationen durchgesucht und diese in einer Liste für die Kaskade zur Verfügung gestellt. Diese arbeitet sie dann nach folgenden Kriterien ab.

3.6.2 Herkunft und Wichtigkeit

Die Herkunft einer Deklaration basiert darauf, woher sie stammt, und ihre Wichtigkeit hängt davon ab, ob sie mit **!important** (siehe Abschnitt 3.2) deklariert ist oder nicht.

Die Rangfolge der verschiedenen Ursprünge und Wichtigkeiten lautet in aufsteigender Reihenfolge, also von der unwichtigsten bis zur wichtigsten Herkunft:

1. **Deklaration durch das *Browser-Stylesheet*:** Browser geben für einige Elemente ein Standard-Aussehen vor, etwa die Größe der einzelnen Überschriften etc.
2. **Deklaration durch das *User-Stylesheet*:** Benutzer*innen können beim Browser ein eigenes Stylesheet hinterlegen, das z. B. auf allen Seiten die Schrift prinzipiell größer darstellt.
3. **Deklaration durch das *Author-Stylesheet*:** Das Stylesheet, das zusammen mit der Website erstellt wird.
4. **Deklaration aus *CSS-Animationen*:** Animierte Werte werden berechnet und müssen eine höhere Priorität als normale Regeln haben, damit sie immer korrekt funktionieren.
5. ***Important-Deklaration im Author-Stylesheet*:** Alle Style-Angaben mit dem Schlüsselwort `!important` werden als wichtiger betrachtet.
6. ***Important-Deklaration im User-Stylesheet*:** Noch mehr Gewicht, um für sich persönlich im Browser Angaben endgültig überschreiben zu können.
7. ***Important-Deklaration im Browser-Stylesheet*:** In Browser-Stylesheets werden mit `!important`-Regeln Dinge gesetzt, die nicht überschrieben werden können, um böses Verhalten zu verhindern. So ist es Webseiten-Autor*innen etwa nicht möglich, die Maskierung bei Passwortfeldern nicht entfernen.
8. **Deklaration aus *CSS-Transitions*:** Übergänge zwischen CSS-Werten. Diese werden zuletzt berechnet, da sie auf Ergebnissen der Kaskade beruhen.

Arten von Stylesheets

Wie also ersichtlich, gibt es drei Arten von Stylesheets: Das *Browser-Stylesheet*, das *User-Stylesheet* und das *Author-Stylesheet*. Wichtig für die Lehrveranstaltung ist das *Author-Stylesheet*, welches im weiteren Verlauf dieses Skriptums nur mehr schlicht als „Stylesheet“ bezeichnet wird. Ist vom Browser- oder User-Stylesheet die Rede, so werden diese explizit so benannt.

Browser-Stylesheet: Werden in einem HTML-Dokument keine CSS-Angaben verwendet, gibt der Browser das Aussehen in einigen Punkten durch ein so genanntes Browser-Stylesheet (auch Default-Stylesheet oder User Agent Stylesheet) vor. Dieses wird im HTML Living Standard [8, Abschnitt 15] spezifiziert. Browserhersteller sind dazu angehalten, dies genauso zu implementieren². Daher sehen unformatierte Seiten in fast jedem Browser gleich aus.

Da das Browser-Stylesheet in der Herkunftsreihenfolge der Kaskade die niedrigste Priorität erhält, sind alle anderen Style-Angaben wichtiger und überschreiben daher diejenigen im Browser-Stylesheet. Das Browser-Stylesheet ist jedoch meist die Ursache für Formatierungen, die nicht explizit gesetzt wurden (z. B. vertikaler Abstand vor

²Hier ist etwa das Stylesheet, das in Chrome/Chromium zum Einsatz kommt: <https://chromium.googlesource.com/chromium/blink/+refs/heads/main/Source/core/css/html.css>

und nach den diversen Überschriften). Dies sollte immer im Hinterkopf behalten werden. Mit **!important** markierte Regeln aus dem Browser-Stylesheet haben jedoch (mit Ausnahme von Transitions) das höchste Gewicht. Dies nützen die Browser um einige sicherheitskritische Formatierungen zu definieren, die nicht überschrieben werden sollen.

User-Stylesheet: User-Stylesheets ermöglichen Benutzer*innen, eigene Formatierungen vorzunehmen, die sich auf alle im Browser betrachteten Webseiten beziehen. In der Regel wird diese Möglichkeit weniger genutzt, Personen mit schlechter Sehkraft können aber etwa in einem User-Stylesheet die Schriftgröße anpassen. Ein anderes Szenario wäre, mit Hilfe des User-Stylesheets Schrift- und Hintergrundfarbe neutral zu gestalten.

In Mozilla Firefox lässt sich dies durch Änderungen in der Datei `userContent.css` bewerkstelligen, welche sich im Profil-Ordner befindet, wenn dies zuvor in den erweiterten Einstellungen aktiviert wurde. Andere Browser, wie etwa Google Chrome, haben die Unterstützung für User-Stylesheets bereits seit längerem eingestellt³, es gibt jedoch Extensions wie Stylus⁴, die dafür verwendet werden können.

3.6.3 Kontext

Dieser Schritt ist wichtig für fortgeschrittenere CSS-Techniken wie Webkomponenten. CSS lässt mehrere Kontexte zu, so kann eine Seite ihre normalen CSS-Regeln beinhalten (äußerer Kontext) und dann eine externe Komponente (z. B. ein Kalender-Widget) inkludieren, die ihr eigenes CSS mitbringt, mit dem sie formatiert wird (innerer Kontext). Um zu ermöglichen, dass dieses Kalender-Widget im Stile der Seite angezeigt wird, haben die Regeln der Seite, also der äußere Kontext, Vorrang gegenüber dem inneren (den Regeln des Widgets). Bei **!important**-Regeln gewinnen jedoch die Regeln des inneren Kontexts, damit dieser Style-Angaben setzen kann, die von außen nicht überschrieben werden können.

3.6.4 Element-Attached-Styles (Inline-Styles)

Eine CSS-Regel, die direkt bei einem HTML-Element (mit dem `style`-Attribut, siehe Abschnitt 3.3.1) steht, schlägt immer eine CSS-Regel derselben Wichtigkeit aus einem Stylesheet.

3.6.5 Layers

Mit der Regel `@layer` können CSS-Regeln in eigene Schichten eingeteilt werden, was innerhalb einer Herkunft nochmals eine feinere Unterteilung ermöglicht.

Werden etwa mit

```
1 @layer reset, base, components, utilities;
```

vier Schichten definiert, so gewinnen die CSS-Regeln des weiter hinter deklarierten Layers. Regeln im `utilities`-Layer haben also mehr Gewicht als die in `components` usw.

³<https://issues.chromium.org/issues/41091633>

⁴<https://chromewebstore.google.com/detail/stylus/clngdbkpkpeebahjckkfobafhncgmne>

Für Regeln mit `!important` ist es genau umgekehrt. Hier schlagen die Angaben in `reset` die aus `base` usw.

Layers sind ein sehr neues und mächtiges Konzept, das in Vorlesung 5 etwas detaillierter vorgestellt wird.

3.6.6 Specificity (Gewichtung)

Die *Spezifität* (engl. „specificity“) besagt, dass im Falle von zwei im selben Dokument für dasselbe Element spezifizierten Regeln die spezifischere zum Zug kommt. Ein Beispiel wären die folgenden beiden Regeln:

```
1 body h1 {  
2   color: green;  
3 }  
4 h1 {  
5   color: red;  
6 }
```

Beide formatieren `<h1>`-Elemente (einmal rot, einmal grün). Wie lässt sich nun sagen, welche Regel spezifischer ist? CSS sieht dafür einen einfachen, numerischen Mechanismus vor, mit dem sich die Specificity ausrechnen lässt.

Jeder Selektor besitzt ein Gewicht, das in einer dreiteiligen Form beschrieben wird: 0,0,0. Für jeden Teil des Selektors werden nun bestimmte Werte (Gewicht) hinzugezählt:

- ID-Selektoren zählen 1,0,0,
- Klassenselektoren, Pseudoklassen und Attributselektoren zählen 0,1,0,
- Elemente und Pseudoelemente zählen 0,0,1,
- der Universalselektor hat keine Auswirkung, zählt also 0,0,0.

Die funktionalen Pseudoklassen `:is()`, `:has()` und `:not()` selbst tragen nicht zum Gewicht des Selektors bei, sondern ihre Inhalte. Dabei wird zählt nur die Angabe mit der höchsten Gewichtung. `:is(h1, .red, #content)` etwa zählt 1,0,0 (aufgrund des ID-Selektors). Die Ausnahme ist die funktionale Pseudoklasse `:where()`. Sie zählt wie der Universalselektor immer 0,0,0 (unabhängig von ihrem Inhalt).

Anmerkung: Angaben mit dem `style`-Attribut im HTML-Markup sind nicht (mehr) Teil der Specificity-Berechnung, sondern in einer eigenen Ebene der Kaskade (Element-Attached-Styles, Abschnitt 3.6.4) und haben immer höheres Gewicht. Historisch wird dies manchmal noch mit einer vierten Gewichtsangabe dargestellt: 1,0,0,0.

CSS-Regeln mit dem `!important`-Schlüsselwort unterliegen nicht der Specificity-Berechnung mit „normalen“ Regeln, da auch hier im Punkt „Reihenfolge“ in der Kaskade ein anderer Abschnitt zum Zug kommt. Gibt es aber mehrere Regeln mit `!important`-Angaben, die dasselbe Element betreffen, so wird zwischen diesen ebenso die Specificity berechnet.

Für die obigen Angaben lässt sich damit folgendermaßen rechnen:

- `body h1 { color: green; }`: $2 \times$ Elementselektor ergibt 0,0,2.
- `h1 { color: red; }`: $1 \times$ Elementselektor ergibt 0,0,1.

Somit ist die erste Regel spezifischer (da der Wert höher ist) und kommt zum Zug.

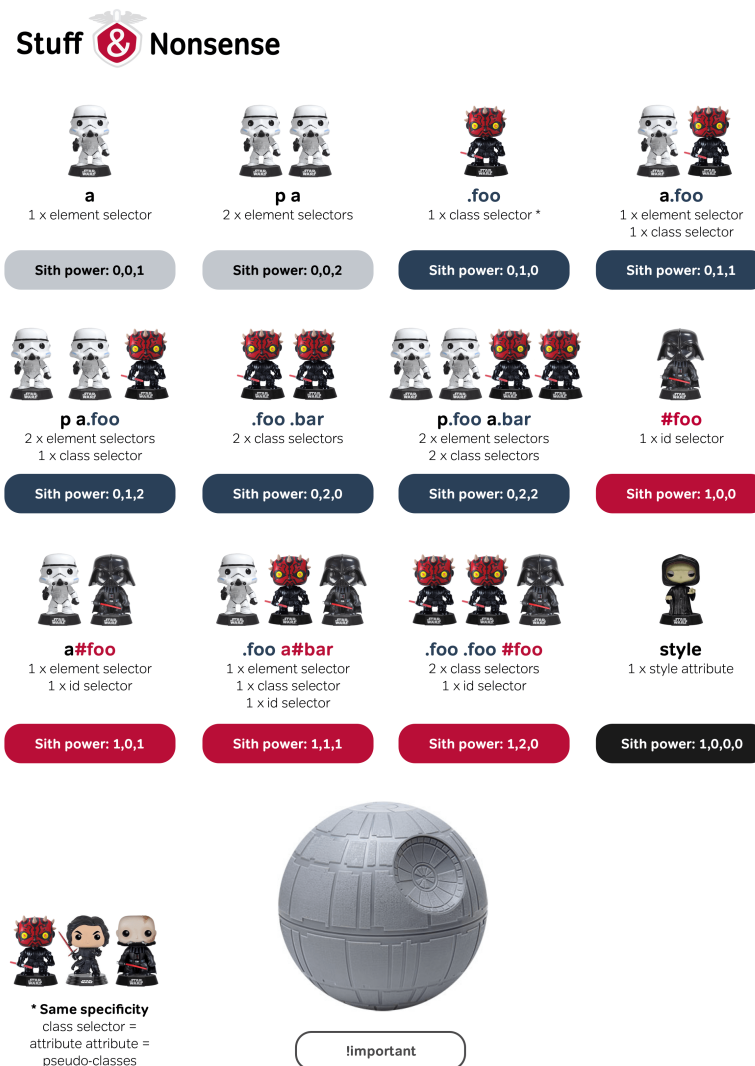


Abbildung 3.14: Je spezifischer ein Selektor, desto höher die Sith-Power: Storm Trooper (0,0,1), Darth Maul (0,1,0), Darth Vader (1,0,0), Emperor (alte Syntax für Inline-Styles 1,0,0,0). Der Todesstern (!important) zerstört alles, da dadurch eine andere Ebene in der Kaskade erreicht wird. Bildquelle: [1].

Vorsicht: Die vier Stellen der Specificity verhalten sich nicht wie eine natürlich Zahl im 10er-System. Würde etwa eine Specificity von 0,0,9 (durch etwa 9 Typselektoren in einer Angabe) bestehen und würde in die CSS-Regel ein weiterer Elementselektor hinzugefügt werden, so ergäbe sich eine Specificity von 0,0,10 und nicht 0,1,0. Somit ist eine CSS-Klasse immer höherwertiger als 10 Elementselektoren.

Abbildung 3.14 stellt einige weitere Beispiele grafisch dar.

Unter <https://specificity.keegan.st/> gibt es einen Specificity-Rechner, der bei Angabe des CSS-Selektors die Specificity errechnet.

3.6.7 Reihenfolge

Sind zwei Angaben nach diesen Regeln immer noch gleichwertig (gleiche Herkunft, gleicher Kontext, gleicher Layer, gleiche Specificity), so „gewinnt“ die später definierte (also im Stylesheet „weiter unten“ angegebene).

3.7 Werte an CSS-Eigenschaften übergeben

CSS-Eigenschaften akzeptieren unterschiedliche Arten von Werten. Dies können numerische Angaben, Zeichenketten, Schlüsselwörter, Farben oder auch Winkelmaße sein.

3.7.1 Numerische Angaben

Eigenschaften, die numerische Angaben benötigen (wie die Schriftgröße oder die Breite eines Elements) akzeptieren eine Reihe an unterschiedlichen Einheiten. Unterschieden wird zwischen absoluten und relativen Werten.

Absolute Werte

Folgende *absolute Werte* sind in CSS zulässig:

- **mm**: Millimeter,
- **cm**: Zentimeter (1 cm = 100 mm),
- **in**: Zoll (Inch, 1 in = 2,54 cm),
- **pt**: Punkt (Point, 1 pt = 1/72 in),
- **pc**: Pica (1 pc = 12 pt),
- **px**: Pixel (1 px = 0,75 pt).

Gebräuchlich sind **px** bei Breitenangaben von Elementen (da der Bezug die Bildschirmauflösung ist und so nicht umgerechnet werden muss) und Schriftgrößen im Bezug auf die Bildschirmdarstellung. Bei Stylesheets für den Druck sind in der Regel die anderen Maßeinheiten angebracht. Pixel sind genau genommen auch eine relative Einheit, da Pixel sehr stark vom Ausgabegerät und dessen Pixeldichte und Auflösung abhängen.

Relative Werte

Zu den *relativen Angaben* zählen:

- **em**: Bezieht sich auf die Schriftgröße (**font-size**), die für das aktuelle Element eingestellt ist. Diese kann explizit gesetzt oder vererbt sein. Hat etwa ein Absatz die Schriftgröße **12px** vererbt bekommen (weil sie z. B. im für `<body>` gesetzt wurde) und wird dann die Regel `p { font-size: 3em }` gesetzt, so hat der Absatz eine Größe von **36px**.
- **rem**: Funktioniert wie **em**, bezieht sich jedoch immer auf die Schriftgröße des Wurzelements (**rem** bedeutet „root em“) und erlaubt daher eine konstante Verwendung von **em**-Werten im Stylesheet.
- **ex**: Diese Einheit ist durch die erste für das Element verfügbare Schriftart definiert und bezieht sich auf deren Höhe des Buchstabens „x“. Diese Größe ist in der Regel bei der Schriftart selbst hinterlegt.

- **ch**: Diese Einheit definiert sich über die Breite des Zeichens „0“.
- **lh**: Die Zeilenhöhe des Elements.
- **rlh**: Die Zeilenhöhe des Wurzelements (analog zu **rem**).
- **vw**: Entspricht 1 % der Anzeigenbreite (viewport width), also der Breite des Browserfensters.
- **vh**: Entspricht 1 % der Anzeighöhe (viewport height), also der Höhe des Browserfensters.
- **vmin**: Entspricht 1 % der kleineren Seite der Anzeige (viewport min).
- **vmax**: Entspricht 1 % der größeren Seite der Anzeige (viewport max).
- **vb** und **vi**: Viewport Block und Viewport Inline Eineinheiten. Entsprechen je 1 % der Breite bzw. Höhe, je nach eingestellter Schreibrichtung. Bei horizontaler Richtung ist 1 **vb** gleich 1 **vh** und 1 **vi** gleich 1 **vw**.
- **svw** und **svh**: Wie **vw** und **vh** nur für den „small viewport“, also der kleinstmögliche Anzeigebereich bei dem der Platz für die User-Interface-Elemente des Browsers schon abgezogen wurde.
- **lvw** und **lvh**: Wie **vw** und **vh** nur für den „large viewport“, also der größtmögliche Anzeigebereich. Hier könnten Inhalte aber durch UI-Elemente verdeckt werden.
- **dvw** und **dvh**: Wie **vw** und **vh** nur für den „dynamic viewport“. Hier können sich die Größen ändern, wenn UI-Elemente verschwinden bzw. erscheinen.
- **%**: Prozentangaben funktionieren ähnlich wie die **em**-Angaben, nur ist der Bezugspunkt nicht nur die Schriftgröße, es können z. B. auch Längenangaben eines übergeordneten Elements sein.

Gebräuchlich sind hier vor allem die Angaben in **em**, **rem** und **%**, auch die Viewport-bezogenen Angaben nehmen stark an Bedeutung zu. Relative Angaben sind vor allem dann vorteilhaft (oder überhaupt notwendig), wenn sich das Stylesheet auf unterschiedlichen Ausgabegeräten (Stichwort: responsive Web) funktionieren soll.

3.7.2 Zeichenketten und Schlüsselwörter

Zeichenketten, oder vereinfacht ausgedrückt Text, muss in CSS in einfache oder doppelte Anführungszeichen gesetzt werden. Gebraucht wird dies etwa, um Text mit der **:before** Pseudoklasse einzufügen:

```
1 p:before {  
2   content: "Text der davor eingefügt wird";  
3 }
```

Viele CSS-Eigenschaften hingegen verlangen Schlüsselwörter. Dies sind vordefiniert und dürfen *nicht* unter Anführungszeichen gesetzt werden. Beispiele sind etwa Farban-gaben oder das Schriftgewicht.

```
1 p {  
2   color: blue;  
3   font-weight: bold;  
4 }
```

3.7.3 Farben

Farben können in CSS auf mehrere Arten definiert werden. Einige davon waren bereits in den vorangehenden Abschnitten im Einsatz. Diese sind:

- benannte Farben (Schlüsselwörter),
- Hexadezimalschreibweise,
- RGB-Angabe,
- HSL-Angabe,
- Lab- und Oklab-Angaben,
- LCh- und Oklch-Angaben,
- HWB-Angabe.

Benannte Farben

Der CSS 2.1 Standard [11] definiert 17 verschiedene Schlüsselwörter für Farben, nämlich **black, maroon, green, olive, navy, purple, teal, silver, gray, red, orange, lime, yellow, blue, fuchsia, aqua** und **white**. In der Color Module Level 4 Spezifikation [18] werden noch weitere 133 Schlüsselwörter definiert, sodass nun insgesamt 150 zulässige Schlüsselwörter für Farben existieren. Eine Übersicht findet sich bei MDN⁵.

```
1 body {  
2   color: rebeccapurple;  
3 }
```

Hexadezimalschreibweise

Seit der ersten CSS-Version können Farben in Hexadezimalschreibweise notiert werden. Hinter dem Rautezeichen (#) folgen drei Hexadezimalwerte zwischen 00 (für 0) und FF (für 255). Diese spezifizieren den jeweiligen RGB (rot, grün, blau) Anteil.

Im folgenden Beispiel wird einmal die Farbe weiß für die Schriftfarbe und die Farbe blau für den Hintergrund definiert.

```
1 section {  
2   color: #FFFFFF;  
3   background-color: #0000FF;  
4 }
```

Sind die erste und zweite Ziffer einer Farbangabe identisch, kann eine verkürzte Schreibweise verwendet werden. So steht #FFF für #FFFFFF oder #00F für #0000FF.

RGB-Angabe

RGB-Farbangaben können auch mit einer Dezimal- oder Prozentschreibweise erreicht werden. Dafür steht die CSS-Funktion **rgb()** zur Verfügung. Eingetragen werden Zahlen von 0 bis 255. So wird das Beispiel mit blauem Hintergrund und weißer Schriftfarbe wie folgt geschrieben. Der Hintergrund ist dabei zu 50 % Transparent – der Alphawert wird zwischen 0.0 (völlig transparent) und 1.0 (völlig opak) angegeben und durch einen Schrägstrich getrennt.

⁵<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/named-color>

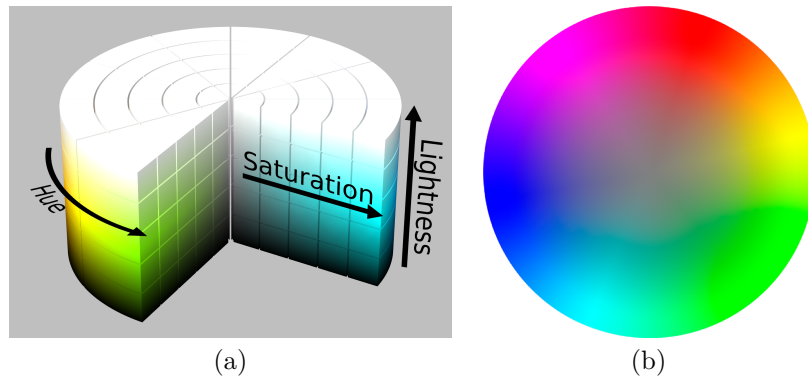


Abbildung 3.15: Der HSL-Zylinder (a) und der zugehörige Farbkreis (b). Hue ist der Winkel am Farbkreis, Saturation ist der Radius, Lightness ist die Höhe des Zylinders. Rot liegt bei 0°, Gelb bei 60°, Grün bei 120°, Cyan bei 180°, Blau bei 240° und Magenta bei 300°. Quellen: [3, 7]

```
1 section {
2   color: rgb(255 255 255);
3   background-color: rgb(0 0 255 / 0.5);
4 }
```

Alternativ können die Farbangaben auch in Prozentangaben erfolgen. Weiß wäre somit `rgb(100% 100% 100%)`.

HSL-Angabe

HSL steht für *Hue*, *Saturation*, *Lightness*, das Farbsystem wird durch einen Zylinder repräsentiert (Abbildung 3.15 (a)). Hue repräsentiert die Grundfarbe aus dem Farbkreis (Abbildung 3.15 (b)), Saturation gibt die Sättigung über den Radius an, je höher der Wert, desto größer ist der Radius. Schließlich definiert Lightness die Helligkeit. 50 % entspricht dabei der normalen Helligkeit, 100 % entspricht immer Weiß, 0 % immer Schwarz. Eine HSL-Farbdefinition ist mit dem Schlüsselwort `hsl()` möglich, eine optionale Angabe der Transparenz erfolgt mittels Schrägstrich.

Anbei die Farbangaben für weiße Schrift und blauen, halbtransparenten Hintgrund:

```
1 section {
2   color: hsl(0 0% 100%);
3   background-color: hsl(240 100% 50% / 0.5);
4 }
```

Lab-, Oklab, LCh-, Oklch und LWB-Angaben

In der CSS Color Module Level 4 Spezifikation [18] wurden neuen Möglichkeiten zur Farbspezifikation eingeführt.

- Mit `lab()` können CIELAB-Farben im Lab-Farbraum (Helligkeit, rot-grün-Wert, gelb-blau-Wert) definiert werden.
- Mit `oklab()` wird selbiges für den etwas moderneren Oklab-Farbraum definiert.

- `lch()` erlaubt es CIELAB-Farben mittels Lightness, Chroma und Hue-Werten anzugeben.
- `oklch` ermöglicht selbiges für den Oklab-Farbraum.
- Mit `hwb()` werden Farben durch Hue, Whiteness und Blackness definiert.

3.7.4 Winkelmaße und Zeitangaben

Für CSS-Transformationen (z. B. Rotation) werden Winkelmaße benötigt, für Zeitangaben stehen ebenfalls zwei Einheiten zur Verfügung.

Winkelmaße

Winkelangaben können in vier verschiedenen Einheiten angegeben werden:

- Grad: Mit der Einheit `deg` werden Grad angegeben. Ein Kreis entspricht 360 Grad (`360deg`).
- Gon: Mit `grad` werden Gon angegeben, ein Kreis entspricht 400 Gon (`400grad`).
- Radian: Das Bogenmaß wird mittels `rad` spezifiziert. Ein Kreis entspricht $2 \cdot \pi$, also gerundet 6.28 (`6.28rad`).
- Vollwinkel: Die Einheit `turn` steht für eine Kreisumrundung, ein Kreis ist also `1turn`.

Mit dem folgenden Code wird ein `<div>`-Element um 90 Grad rotiert (Details zu Transformationen siehe [25]).

```
1 div {  
2   rotate: 90deg;  
3 }
```

Zeitangaben

In zeitbasierten Formatierungen (wie für Animationen oder Übergänge) werden Angaben für die Zeitdauer benötigt. Zur Verfügung stehen Sekunden (`s`) oder Millisekunden (`ms`).

Im folgenden Code wird ein Übergang 2 Sekunden verzögert.

```
1 a {  
2   transition-delay: 2s;  
3 }
```

3.8 Erste CSS-Formate

Nachdem in den vorigen Abschnitten aus Gründen der praktischen Erklärung schon einige Eigenschaften vorgestellt wurden, folgen hier nun exemplarisch einige ausführlicher erklärte CSS-Formate. Weitere können selbst in diversen Dokumentationen [5] nachgelesen werden.

Aufgrund der Menge an Selektoren wird nicht bei jedem ein Browser-Screenshot abgebildet.

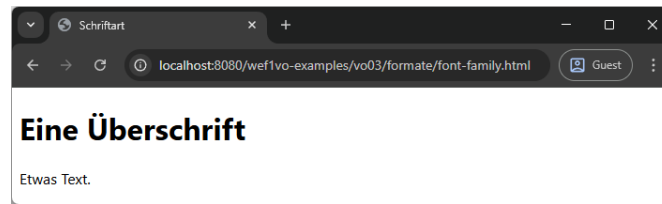


Abbildung 3.16: Per `font-family`-Regel wurde die Schriftart für das ganze Dokument geändert.

3.8.1 Schriften-Formatierung

Unter *Schriften-Formatierung* fallen alle Dinge, die mit Schriftarten, -größe und -stilen zu tun haben. Die dazu gehörigen CSS-Eigenschaften beginnen alle mit **font**.

Schriftart

Die *Schriftart* (engl. „font-family“) gibt die verwendete Schriftfamilie an. Die CSS-Eigenschaft lautet **font-family**. Dabei können nur Schriften verwendet werden, die auch lokal im System der jeweiligen Benutzer*innen installiert sind (z. B. Helvetica Neue oder Arial). Aus diesem Grund ist es ratsam, gleich mehrere Schriften durch Komma getrennt anzugeben. Findet der Browser die erste angegebene Schrift nicht, wird auf die nächste gewechselt usw. Als letzte Option sollte immer eine vordefinierte Schrift verwendet werden, die auf jeden Fall erkannt wird (diese lauten **serif**, **sans-serif**, **monospace**, **cursive**, **fantasy**, **system-ui**, **ui-serif**, **ui-sans-serif**, **ui-monospace**, **ui-rounded**, **math** und **fangsong**). Findet der Browser keine Schrift aus der angegebenen Liste, so wählt er selbst eine Schrift aus. Schriftarten, deren Namen Leer- oder andere Spezialzeichen enthalten, sollten unter Anführungszeichen gestellt werden. Gerade die **ui**-Varianten werden mittlerweile gerne vorab verwendet, um gezielt die Systemschriften zu verwenden.

Folgende Regel definiert für das gesamte Dokument (`<body>`) drei verschiedene System-Schriften, sowie eine Fallback-Option. Das Ergebnis ist in Abbildung 3.16 zu sehen:

```
1 body {  
2   font-family: system-ui, "Segoe UI", Roboto, Arial, sans-serif;  
3 }
```

Schriftgröße

Die *Schriftgröße* (engl. „font-size“) wird mit der Eigenschaft **font-size** definiert. Möglich sind numerische Angaben (sowohl absolut als auch relativ) sowie die vordefinierten Werte **xx-small**, **x-small**, **small**, **medium**, **large**, **x-large**, **xx-large**, **smaller** und **larger**.

Anbei eine Regel, die die Schriftgröße des gesamten Dokuments auf 16 Pixel setzt.

```
1 body {  
2   font-size: 16px;  
3 }
```

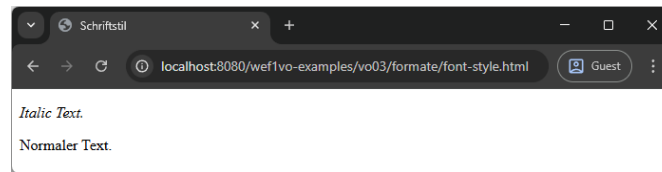


Abbildung 3.17: Der erste der beiden Absätze wird kursiv dargestellt.

Schriftgewicht

Das *Schriftgewicht* (engl. „font-weight“) bestimmt die Schriftstärke, also ob eine Schrift leicht, normal oder fett dargestellt wird. Es haben sich dabei 9 Stufen eingebürgert: Ultraleicht, extraleicht, leicht, halbleicht, normal, halbfett, fett, extrafett und ultrafett. Mit Hilfe der CSS-Eigenschaft **font-weight** lassen sich diese Stufen angeben. Verwendet werden dazu die numerischen Angaben von 100 bis 900 (in 100er Schritten), wobei 100 ultraleicht und 900 ultrafett entspricht. Darüber hinaus gibt es noch die Angaben **normal** (entspricht 500), **bold** (entspricht 700), **bolder** (entspricht 800) und **lighter** (entspricht 200). Der Browser entscheidet, wie er die Angaben den verfügbaren Schriftschnitten zuweist.

Folgende CSS-Regel stellt einen Absatz fett dar:

```
1 p {  
2   font-weight: bold;  
3 }
```

Schriftstil

Der *Schriftstil* (engl. „font-style“) bestimmt die Schriftlage, also ob der normale, kursive oder schräggestellte Schriftschnitt verwendet werden soll. Möglich ist dies mit der CSS-Eigenschaft **font-style**, verfügbare Werte sind **normal**, **italic** oder **oblique**. Bei **italic** wird, wenn verfügbar, der kursive Schriftschnitt einer Schriftart verwendet. **oblique** stellt lediglich den normalen Schriftschnitt schräg und sollte daher nur in begründeten Fällen verwendet werden.

Folgende CSS-Regel erstellt eine CSS-Klasse, die kursive Schrift erzeugt, in Abbildung 3.17 wird das Ergebnis dargestellt.

```
1 .important {  
2   font-style: italic;  
3 }
```

Schrift-Variante

Mit Hilfe der *Schrift-Variante* (engl. „font-variant“) kann Kapitälchen-Text erzeugt werden, also Text bei dem für die Kleinbuchstaben kleinere Großbuchstaben verwendet werden (wie z. B. bei den Wörtern WEB FUNDAMENTALS). Verantwortlich ist die Eigenschaft **font-variant** mit dem Wert **small-caps**. Der zweite mögliche Wert ist **normal**.

Durch die folgende CSS-Regel wird Text in `<em>`-Elementen als Kapitälchen dargestellt.


```

1 em {
2   font-variant: small-caps;
3 }

```

Shorthand-Syntax

Für die gesamte Formatierung von Schriften existiert eine so genannte *Shorthand-Syntax*, also eine Möglichkeit, alle Angaben in eine einzige CSS-Eigenschaft zu verpacken. Diese lautet **font**. Bei der Verwendung müssen Schriftgröße und -art zwingend am Ende der Regel vorkommen. Schriftgewicht, -variante und -stil sind optional und können in beliebiger Reihenfolge am Beginn der Regel stehen.

Folgender Code definiert alle obigen Eigenschaften in einer Regel:

```

1 body {
2   font: bold italic small-caps 16pt system-ui, "Segoe UI", Roboto, Arial, sans-serif;
3 }

```

Web Fonts

Eine der größten Schwachstellen im Web war seit langem das Kapitel Typographie. Website-Autor*innen waren meist auf einige wenige Systemschriftarten limitiert, die auf dem Client-Rechner installiert sein mussten, um korrekt angezeigt zu werden. Um kein Risiko einzugehen, wurde dabei auf ein kleines Set an Schriften zurückgegriffen, die überall vorhanden, aber keineswegs immer passend waren. Das Modul *CSS Fonts Module Level 3* [19] beinhaltet Möglichkeiten, herunterladbare Schriften zu verwenden.

Zentrales Element der Web Fonts ist die **@font-face**-Regel. Sie gibt zusätzliche Schriften an und definiert, wo diese zu finden sind. Folgende Regel definiert die Schriftart *Noto Sans* und gibt den URL zum Download an, sodass sie vom Webbrowser herunter geladen werden kann.

```

1 @font-face {
2   font-family: "Noto Sans";
3   src: url("noto-sans-v42-latin-regular.woff2") format("woff2");
4 }

```

In dem obigen Beispiel wird die (freie) Schriftart Noto Sans (<https://fonts.google.com/noto>) mit Hilfe der **font-family**-Angabe spezifiziert. Der Name kann prinzipiell frei gewählt werden, sollte aber idealerweise den genauen Schriftartnamen wiedergeben. **src** definiert den Pfad, an dem die Schrift verfügbar ist. In diesem Fall ist es ein relativer Pfad im selben Verzeichnis, es können aber auch absolute Pfade angegeben werden. Die **format**-Angabe am Ende spezifiziert den Typ der Schrift näher, kann aber auch entfallen.

Soll nun ein zweiter Schriftschnitt (z. B. kursiv) angegeben werden, muss eine zweite **@font-face**-Regel angelegt werden:

```

1 @font-face {
2   font-family: "Noto Sans Italic";
3   src: url("noto-sans-v42-latin-italic.woff2") format("woff2");
4 }

```

Beide Schriften sollen nun im folgenden HTML-Fragment zur Anwendung kommen:

```

1 <body>
2   <h1>Überschrift</h1>
3   <p>Erster Absatz.</p>
4   <p>Zweiter Absatz.</p>
5 </body>

```

Der CSS-Code dazu enthält zunächst die `@font-face`-Regeln und wendet die neu definierten Schriften dann wie gewohnt an.

```

1 @font-face {
2   font-family: "Noto Sans";
3   src: url("noto-sans-v42-latin-regular.woff2") format("woff2");
4 }
5 @font-face {
6   font-family: "Noto Sans Italic";
7   src: url("noto-sans-v42-latin-italic.woff2") format("woff2");
8 }
9 h1 {
10  font-family: "Noto Sans", Arial, sans-serif;
11 }
12 p {
13  font-family: "Noto Sans Italic", Arial, sans-serif;
14 }
```

Schriftformate: Zur Verwendung als Web Fonts stehen zwei mögliche *Typen* zur Verfügung:

- **WOFF:** Web Open Font Format – entwickelt von Mozilla, dann aber vom W3C übernommen und mittlerweile der alleinige Standard [29]. Wird von allen Browsern unterstützt und hat sich als der gemeinsame Nenner für Schriften im Web etabliert. Die Dateien sind kleiner und enthalten Zusatzinformationen wie die Lizenz. Die aktuelle Version 2.0 (Erweiterung `.woff2`) hat vor allem den Bandbreitenbedarf bei der Übermittlung optimiert.
- **TrueType/OpenType:** Die bekannten `.ttf`- und `.otf`-Schrifttypen werden in allen Browsern unterstützt, haben aber das Problem, dass keinerlei Lizenzinformationen mitgegeben werden. Es sollten nur jene Schriften verwendet werden, an denen Rechte verfügbar sind und bei denen Weiterverbreitung auf diese Art auch rechtlich gesichert ist.

Font-Dienste: In vielen Fällen ist es nicht nötig, selbst Schriften zu definieren, diese bereit zu stellen und einzubinden, denn es gibt einige *Dienste*, die dies auf eine einfache Art ermöglichen. Viele davon bieten darüber hinaus noch freie Schriften an, die völlig ohne lizenzrechtliche Bedenken verwendet werden können. Der bekannteste ist *Google Fonts* (<https://fonts.google.com/>), es existieren jedoch auch andere Anbieter. Google Fonts stellt die Schriften über `@font-face`-Regeln in eigenen Stylesheets zur Verfügung, die auf der eigenen Seite eingebunden werden können. Weiters wird auch der Download der dort verfügbaren Schriften ermöglicht, die WOFF2-Dateien sind dabei jedoch nicht enthalten. Mit Tools wie dem *google-webfonts-helper* (<https://gwfh.mranftl.com/fonts>) können diese jedoch auch einfach heruntergeladen und selbst gehostet werden.

3.8.2 Text-Formatierung

In den Bereich der Text-Formatierung fallen Dinge wie unterstrichener oder durchgestrichener Text oder auch die Schriftfarbe.

Text-Dekoration

Mit Hilfe der *Text-Dekoration* (engl. „text-decoration“) können Streichungen aller Art erzeugt werden. Verantwortlich dafür ist die CSS-Eigenschaft `text-decoration`. Sie kann die Werte `underline` (unterstrichen), `overline` (überstrichen), `line-through` (durchgestrichen), `blink` (blinkend) und aber auch `none` (keine Eigenschaft) annehmen.

Unterstreichungen sind im Fließtext auch in den Printmedien kaum üblich, sie sollten daher sparsam eingesetzt werden. Diese Eigenschaft kann aber auch dazu verwendet



Abbildung 3.18: Im HTML-Markup ist der Text mit normaler Großschreibung geschrieben, durch die CSS-Regel wird Versalschrift erzeugt.

werden, die standardmäßige Unterstreichung von Hyperlinks zu entfernen.

Im folgenden Beispiel wird eine Regel zum Durchstreichen eines `<span>`-Elements mit einer bestimmten Klasse erstellt:

```
1 span.strikeout {
2   text-decoration: line-through;
3 }
```

Text-Transformation

Mit Hilfe der *Text-Transformation* (engl. „text-transform“) ist es möglich, die Groß- und Kleinschreibung eines Texts festzulegen, unabhängig davon wie dieser im HTML-Markup steht. Dies geschieht mit Hilfe der Eigenschaft `text-transform`. Sie kann die Werte `uppercase` (alles in Großbuchstaben), `lowercase` (alles in Kleinbuchstaben), `capitalize` (nur Wortanfänge groß) oder `none` (keine Transformation) annehmen.

Die folgende CSS-Regel stellt jeglichen Text in allen Absätzen komplett in Großbuchstaben dar, das Ergebnis ist in Abbildung 3.18 zu sehen.

```
1 p {
2   text-transform: uppercase;
3 }
```

Text-Farbe

Die Zuweisung von Farbwerten wurde bereits in Abschnitt 3.7.3 besprochen. Die folgende Regel legt die Standardschriftfarbe für das gesamte Dokument (d. h. für das `<body>`-Element) als Maroon fest.

```
1 body {
2   color: maroon;
3 }
```

3.8.3 Abstände und Ausrichtung

Angaben zu *Abständen* und *Ausrichtung* nehmen Einfluss auf die Art und Weise, wie Absätze im Text angezeigt werden.

Einrückung

Ein in Büchern und Zeitschriften weit verbreitetes Gestaltungselement ist das Einrücken der ersten Zeile in einem mehrzeiligen Text (auch in diesem Skriptum zu sehen). Dies

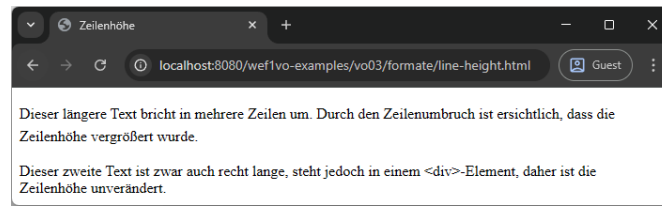


Abbildung 3.19: Der Absatz zeichnet sich durch die größere Zeilenhöhe aus, das `<div>`-Element hat die reguläre Höhe.

kann mit der Eigenschaft `text-indent` erreicht werden. Die Angabe erlaubt wiederum die verschiedensten numerischen Angaben. Negative Werte bewirken eine Ausrückung.

Hier wird die erste Zeile eines Absatzes um die aktuelle Schriftgröße eingerückt.

```
1 p {
2   text-indent: 1em;
3 }
```

Zeilenhöhe

Die Angabe der *Zeilenhöhe* ist vor allem im Zusammenhang mit langem Fließtext interessant, da sie den Abstand zwischen den einzelnen Zeilen bestimmt. Oft kann ein leicht erhöhter Zeilenabstand zur besseren Lesbarkeit beitragen. Mit der Angabe `line-height` kann der Zeilenabstand manipuliert werden. Da diese Angabe nicht zuletzt auch von der Schriftgröße abhängen sollte, sind beide oft gemeinsam zu finden oder wird für die Zeilenhöhe ein relativer Wert mittels `em` Angabe verwendet, der sich direkt auf die Schriftgröße bezieht.

Im folgenden Beispiel wird eine Zeilenhöhe in der 1,5fachen Schriftgröße für Absätze definiert, Abbildung 3.19 zeigt die Darstellung im Browser.

```
1 p {
2   line-height: 1.5em;
3 }
```

Vertikale Ausrichtung

Wird einem Blockelement eine fixe Höhe zugewiesen (mehr dazu in Vorlesung 4) so ist oftmals sehr viel horizontaler Platz vorhanden. Um dann zu entscheiden, ob Text in diesem Element oben, unten oder in der Mitte platziert wird, gibt es die Eigenschaft `vertical-align`. Sie kann die Werte `top` (obenbündig), `middle` (mittig), `bottom` (untenbündig), `baseline` (an der Grundlinie ausrichten), `sub` (tiefer gestellt, ohne die Schriftgröße zu reduzieren), `super` (hochgestellt, ohne die Schriftgröße zu reduzieren), `text-top` (am oberen Schriftrand ausrichten) oder `text-bottom` (am unteren Schriftrand ausrichten) annehmen.

Folgende CSS-Klasse richtet die Schrift vertikal zentriert aus:

```
1 .vcenter {
2   vertical-align: middle;
3 }
```

Horizontale Ausrichtung

Etwas eingängiger als die vertikale Ausrichtung ist die horizontale. Sie bestimmt, wie Fließtext innerhalb eines Absatzes angeordnet wird. Die Eigenschaft **text-align** kann dabei die Werte **left** (linksbündig), **center** (zentriert), **right** (rechtsbündig) oder **justify** (Blocksatz) annehmen.

Im folgenden Beispiel wird der Text in allen Absätzen als Blocksatz angeordnet.

```
1 p {  
2   text-align: justify;  
3 }
```

Textumbruch

Umbrüche im Fließtext werden normal vom Browser selbst aufgrund der vorgegebenen Breite des Browserfensters durchgeführt. Soll dies beeinflusst werden, kann mit der Eigenschaft **white-space** ein anderes Verhalten herbei geführt werden. Folgende Werte sind möglich: **normal** (automatischer Zeilenumbruch), **pre** (Zeilenumbruch wie im Quellcode angegeben – wie beim `<pre>`-Element), **pre-wrap** (wie **pre**, nur erfolgt ein Umbruch, wenn die Breite der Box zur Anzeige nicht mehr ausreicht), **pre-line** (mehrere Whitespaces werden wie bei **normal** zusammengefasst, es erfolgt aber ein Zeilenumbruch wie bei **pre-wrap**) und **nowrap** (kein automatischer Umbruch, nur manuell durch `<br>`).

Die folgende CSS-Klasse deaktiviert alle automatischen Zeilenumbrüche.

```
1 .nowrap {  
2   white-space: nowrap;  
3 }
```

3.8.4 Hintergründe

Hintergründe erlauben es, das Standard-Weiß durch eine andere Farbe oder ein Bild zu ersetzen. Sinnvoll ist die Angabe meist nur bei Blockelementen, Inline-Elemente (Phrasing content) können jedoch genauso einen Hintergrund zugewiesen bekommen.

Hintergrundfarbe

Die *Hintergrundfarbe* wird über die Eigenschaft **background-color** gesetzt. Als Werte sind wiederum die in Abschnitt 3.7.3 angegebenen Farben möglich. Der Standardwert ist **transparent**, der auch explizit gesetzt werden kann, um eine Hintergrundfarbe wieder zu entfernen.

Im Sinne der Accessibility (mehr dazu in Vorlesung 6) sollte darauf geachtet werden, dass stets ein ausreichender Kontrast zwischen Schriftfarbe und Hintergrundfarbe besteht.

Folgende Regel setzt für das gesamte Dokument die Hintergrundfarbe Aqua:

```
1 body {  
2   background-color: aqua;  
3 }
```

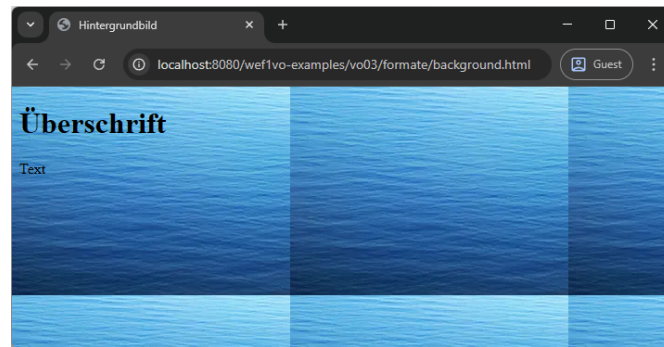


Abbildung 3.20: Da das Hintergrundbild kleiner als die Dimensionen des Browserfensters ist, wird es wiederholt.

Hintergrundbilder

Hintergrundbilder können mit Hilfe der Eigenschaft **background-image** angegeben werden. Als Wert muss ein URL über **url("URL")** übergeben werden. Das eingebundene Hintergrundbild wird im Wallpaperereffekt wiederholt, sofern es nicht die volle Größe ausfüllt. Dieses Verhalten kann mit Hilfe von **background-repeat** angepasst werden (s.u.).

Folgende CSS-Regel legt ein Hintergrundbild auf das gesamte Dokument. Das Hintergrundbild liegt im gleichen Verzeichnis wie die CSS-Datei, Abbildung 3.20 zeigt das Ergebnis.

```
1 body {  
2   background-image: url("background.webp");  
3 }
```

Es ist auch möglich, mehr als nur ein Hintergrundbild zu spezifizieren. Dazu müssen mehrere **url()**-Angaben mit Beistrich getrennt angegeben werden. Kommt es dabei zu Überlagerungen, so wird das erste angegebene Bild ganz oben, das letzte auf der untersten Ebene angezeigt.

Wiederholungseffekt

Mit der Eigenschaft **background-repeat** kann gesteuert werden, ob und wie ein Hintergrundbild wiederholt wird, wenn seine Größe nicht ausreicht, den gesamten Container zu befüllen. Zur Verfügung stehen die Werte **repeat** (Standardeinstellung, Wiederholung in *x*- und *y*-Richtung), **repeat-x** (nur in *x*-Richtung, also horizontal wiederholen), **repeat-y** (nur in *y*-Richtung, also vertikal wiederholen) und **no-repeat** (kein Wiederholen).

Im folgenden Codeausschnitt wird ein zuvor definiertes Hintergrundbild für einen **<div>**-Bereich mit einer ID nur horizontal wiederholt.

```
1 div#container {  
2   background-repeat: repeat-x;  
3 }
```

Wasserzeichen-Effekt

Bei längeren Elementen (z. B. ganzen Seiten, deren vertikaler Inhalt größer ist als das Browserfenster und die daher scrollen müssen) kann entschieden werden, ob der Hintergrund mitscrollen oder statisch verweilen soll. Diese Einstellung wird durch die Eigenschaft **background-attachment** ermöglicht und kann die Werte **scroll** und **fixed** (Wasserzeichen-Effekt) annehmen.

Hier wird das dokumentweite Hintergrundbild beim Scrollen fixiert:

```
1 body {  
2   background-attachment: fixed;  
3 }
```

Position des Hintergrunds

Wird mittels **background-repeat** festgelegt, dass ein Hintergrundbild nicht wiederholt, sondern nur ein Mal angezeigt wird, kann mit der Angabe von **background-position** definiert werden, wie der Hintergrund positioniert werden soll. Die Eigenschaft erwartet zwei durch ein Leerzeichen getrennte Angaben für die horizontale und vertikale Position. Erlaubt sind numerische Werte und die Angaben **top** (vertikal obenbündig), **bottom** (vertikal untenbündig), **center** (horizontal oder vertikal zentriert), **left** (horizontal linksbündig) und **right** (horizontal rechtsbündig).

Im folgenden Beispiel wird eine Hintergrundgrafik horizontal zentriert und vertikal 50 Pixel vom oberen Ende positioniert.

```
1 body {  
2   background-position: center 50px;  
3 }
```

Hintergründe formatieren

Mit mehreren Eigenschaften lässt sich weiter am Erscheinungsbild des Hintergrundbildes feilen.

background-clip kann die Werte **border-box**, **padding-box** oder **content-box** annehmen. Je nachdem erstreckt sich das Hintergrundbild hinter den Rahmen, bis zum Rahmen oder nur im Inhaltsbereich. Dies ist vor allem Beim Einsatz von Rahmen mit Transparenzen interessant.

Mit **background-origin** lässt sich steuern, wo ein Hintergrundbild *beginnt*. Mit dem Wert **border-box** beginnt es bereits hinter dem Rahmen, bei **padding-box** innerhalb des Rahmens und bei **content-box** nur im Inhaltsbereich. Im Unterschied zu **background-clip** beginnt das Hintergrundbild aber nur dort auf der linken Seite, es kann auf der rechten Seite trotzdem bis hinter den Rahmen laufen. Um das zu verhindern, wird **background-clip** benötigt.

Mit der Eigenschaft **background-size** lassen sich Hintergrundbilder skalieren. Die Angaben sind zwei Werte in Pixeln oder Prozent, also etwa **background-size: 20px 40px**, um ein Bild auf 20 Pixel Breite und 40 Pixel Höhe zu skalieren. Alternativ existieren die Werte **contain** und **cover**. Ersteres skaliert das Bild unter Beibehaltung des Seitenverhältnisses so, dass sowohl Höhe als auch Breite in den aktuellen Anzeigebereich

passen. Bei zweiterem wird so skaliert, dass das Hintergrundbild den Anzeigebereich komplett abdeckt (ggf. werden jedoch Teile abgeschnitten).

Shorthand-Syntax

Auch für die Arbeit mit Hintergründen gibt es eine *Shorthand-Syntax*, die es ermöglicht, alle Angaben mit einer Eigenschaft zusammenzufassen. Diese lautet **background**, die Reihenfolge der Einzelangaben ist bei diesem Attribut egal.

Folgende Angabe fasst die oben gezeigten in einer zusammen:

```
1 body {  
2   background: aqua url("background.webp") repeat-x fixed center 50px;  
3 }
```

3.8.5 Listen

Mit Hilfe von CSS lassen sich Listen (meist Aufzählungslisten und nummerierte Listen) formatieren. Vor allem die Aufzählungszeichen stehen hierbei im Mittelpunkt.

Listentyp

Der *Listentyp* ist das Zeichen, das vor jedem einzelnen Listeneintrag angezeigt wird. Bei Aufzählungslisten handelt es sich in der Regel um einen Bullet-Point, bei nummerierten Listen sind es arabische Ziffern. Mit der Eigenschaft **list-style-type** kann dies angepasst werden.

Die folgenden Werte stehen für Aufzählungszeichen zur Verfügung: **disc** (gefüllter Kreis, Bullet-Point), **circle** (leerer Kreis) und **square** (rechteckiger Kasten).

Für Nummerierungen sind folgende Werte möglich: **decimal** (arabische Zahlen), **decimal-leading-zero** (arabische Zahlen mit führender Null), **lower-roman** (römische Zahlen, dargestellt als Kleinbuchstaben), **upper-roman** (römische Zahlen, dargestellt als Großbuchstaben), **georgian** (georgische Nummerierung), **armenian** (armenische Nummerierung), **lower-latin** (lateinische Kleinbuchstaben), **upperlatin** (lateinische Großbuchstaben) und **lower-greek** (griechische Kleinbuchstaben).

Darüber hinaus gibt es mit der Option **none** die Möglichkeit, Aufzählungszeichen jeder Art komplett zu entfernen. Dies ist vor allem dann interessant, wenn eine Liste zur Gliederung einer Navigation verwendet wird, denn dabei werden nur selten Aufzählungszeichen benötigt.

Der folgende Code stellt das Aufzählungszeichen einer Liste auf rechteckige Boxen. Es muss das Listenelement (**** oder ****) selektiert werden, nicht der Listeneintrag (****).

```
1 ul {  
2   list-style-type: square;  
3 }
```

Listenposition

Die *Position des Aufzählungszeichens* kann mittels **list-style-position** bestimmt werden. Möglich sind die Werte **inside** und **outside**, wobei letzterer der Standardwert

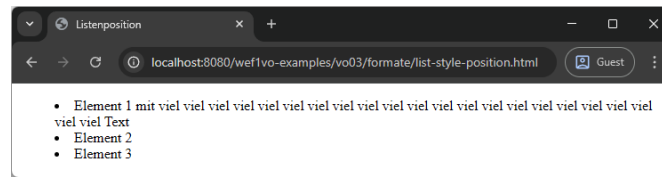


Abbildung 3.21: Beim ersten Aufzählungspunkt mit viel Text ist ersichtlich, dass der Bullet-Point mit dem Text in der Zeile steht.

ist. Wird **inside** gewählt, so beginnt das Aufzählungszeichen mit dem Text, ansonsten steht es davor.

Im folgenden Code wird die Listenposition umgestellt, Abbildung 3.21 zeigt die Darstellung im Browser:

```
1 ul {
2   list-style-position: inside;
3 }
```

Listen-Bilder

Neben der Möglichkeit, andere Aufzählungszeichen zu wählen, können auch Bilder verwendet werden. Dies kann mit der Eigenschaft `list-style-image` bewerkstelligt werden. Sie akzeptiert, ähnlich wie Hintergrundbilder, ebenfalls einen URL auf ein Bild. Dieses wird dann anstatt des regulären Zeichens angezeigt. Die Bildgröße sollte schon im Vorfeld angepasst werden. Obwohl der Browser automatisch versucht, die Größe passend zu wählen, funktioniert dies nicht immer zuverlässig.

Folgende Regel verwendet ein Bild als Aufzählungszeichen für eine Aufzählungsliste.

```
1 ul {
2   list-style-image: url("bullet.webp");
3 }
```

Shorthand-Syntax

Mit der *Shorthand-Syntax* für Listen können ebenfalls wieder die drei einzelnen Angaben zu einer vereint werden. Möglich ist diese mit der Eigenschaft `list-style`. Werden aber die Angaben für `list-style-type` und `list-style-image` gemeinsam angegeben, bekommen die Listen-Bilder Vorrang und werden daher angezeigt.

Folgende Angaben fassen die drei obigen in einer zusammen:

```
1 ul {
2     list-style: square url("bullet.webp") inside;
3 }
```

3.9 Weiterführende Literatur

Das Standardwerk zum Thema Cascading Style Sheets ist [4]. Es bietet eine komplette Einführung mit sehr detailliert erklärten Hintergründen. Die 5. Auflage aus dem Jahr 2023 deckt alle aktuellen Entwicklungen gut ab.

Auf Deutsch gibt es kein alleinstehendes CSS-Werk, das einigermaßen aktuell ist. Hier sollte auf die HTML/CSS-Werke zurückgegriffen werden. In [2, 6, 9, 10] kommen die Themen CSS zusammen mit HTML vor und werden ausreichend erörtert. Ein eigenes CSS-Buch ist daher meist eigentlich nicht nötig, zumal diese Bücher in regelmäßigen Abständen neu aufgelegt werden.

Quellenverzeichnis

- [1] Andy Clarke. *CSS Specificity Wars*. 2018. URL: https://stuffandnonsense.co.uk/archives/css_specificity_wars.html (besucht am 10.11.2025) (siehe S. 3-24).
- [2] Paul Fuchs. *HTML5 und CSS3 für Einsteiger. Der leichte Weg zur eigenen Webseite*. Landshut, Deutschland: BMU Verlag, 21. Juni 2019 (siehe S. 3-41).
- [3] Michael Horvath. *HSL color solid cylinder, alpha, low gamma*. 23. März 2010. URL: https://commons.wikimedia.org/wiki/File:HSL_color_solid_cylinder_alpha_lowgamma.png (besucht am 10.11.2025) (siehe S. 3-28).
- [4] Eric Meyer und Estelle Weyl. *CSS. The Definitive Guide*. 5. Aufl. Sebastopol, USA: O'Reilly, 30. Mai 2023 (siehe S. 3-40).
- [5] Mozilla Developer Network and individual contributors. *Mozilla Developer Network*. Nov. 2025. URL: <https://developer.mozilla.org/en-US/> (besucht am 10.11.2025) (siehe S. 3-29).
- [6] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 3-41).
- [7] Alexandre Van de Sande. *Color circle (hue-sat), trans*. 21. Mai 2023. URL: [https://commons.wikimedia.org/wiki/File:Color_circle_\(hue-sat\).png](https://commons.wikimedia.org/wiki/File:Color_circle_(hue-sat).png) (besucht am 10.11.2025) (siehe S. 3-28).
- [8] Web Hypertext Application Technology Working Group. *HTML. Living Standard*. 4. Nov. 2025. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 3-21).
- [9] Christian Wenz und Christoph Prevezanos. *Jetzt lerne ich HTML5 und CSS3*. 2. Aufl. Burgthann, Deutschland: Markt + Technik Verlag, 16. Nov. 2022 (siehe S. 3-41).
- [10] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 3-41).
- [11] World Wide Web Consortium. *Cascading Style Sheets Level 2 Revision 1 (CSS 2.1) Specification*. W3C Recommendation. Edited in place 12 April 2016 to point to new work, edited in place 07 December 2021 to correct markup. 7. Juni 2011. URL: <https://www.w3.org/TR/CSS2/> (siehe S. 3-2, 3-3, 3-27).
- [12] World Wide Web Consortium. *Cascading Style Sheets Level 2 Revision 2 (CSS 2.2) Specification*. W3C First Public Working Draft. 12. Apr. 2016. URL: <https://www.w3.org/TR/CSS22/> (siehe S. 3-2).

- [13] World Wide Web Consortium. *Cascading Style Sheets, level 1*. W3C Recommendation. Revised 11 Apr 2008, superseded on 13 September 2018. 17. Dez. 1996. URL: <https://www.w3.org/TR/CSS1/> (siehe S. 3-2).
- [14] World Wide Web Consortium. *CSS Animations Level 1*. W3C Working Draft. 2. März 2023. URL: <https://www.w3.org/TR/css-animations-1/> (siehe S. 3-3).
- [15] World Wide Web Consortium. *CSS Backgrounds and Borders Module Level 3*. W3C Candidate Recommendation Draft. 11. März 2024. URL: <https://www.w3.org/TR/css-backgrounds-3/> (siehe S. 3-2).
- [16] World Wide Web Consortium. *CSS Box Model Module Level 3*. W3C Recommendation. 11. Apr. 2024. URL: <https://www.w3.org/TR/css-box-3/> (siehe S. 3-2).
- [17] World Wide Web Consortium. *CSS Cascading and Inheritance Level 5*. W3C Candidate Recommendation Snapshot. 13. Jan. 2022. URL: <https://www.w3.org/TR/css-cascade-5/> (siehe S. 3-3, 3-20).
- [18] World Wide Web Consortium. *CSS Color Module Level 4*. W3C Candidate Recommendation Draft. 25. Apr. 2025. URL: <https://www.w3.org/TR/css-color-4/> (siehe S. 3-2, 3-27, 3-28).
- [19] World Wide Web Consortium. *CSS Fonts Module Level 3*. W3C Recommendation. 20. Sep. 2018. URL: <https://www.w3.org/TR/css-fonts-3/> (siehe S. 3-3, 3-32).
- [20] World Wide Web Consortium. *CSS Images Module Level 3*. W3C Candidate Recommendation Draft. 18. Dez. 2023. URL: <https://www.w3.org/TR/css-images-3/> (siehe S. 3-3).
- [21] World Wide Web Consortium. *CSS Multi-column Layout Module Level 1*. W3C Candidate Recommendation Snapshot. 16. Mai 2024. URL: <https://www.w3.org/TR/css-multicol-1/> (siehe S. 3-3).
- [22] World Wide Web Consortium. *CSS Snapshot 2025*. W3C Group Note. 18. Sep. 2025. URL: <https://www.w3.org/TR/css/> (siehe S. 3-2).
- [23] World Wide Web Consortium. *CSS Syntax Module Level 3*. W3C Candidate Recommendation Draft. 24. Dez. 2021. URL: <https://www.w3.org/TR/css-syntax-3/> (siehe S. 3-2).
- [24] World Wide Web Consortium. *CSS Text Module Level 3*. W3C Candidate Recommendation Draft. 30. Sep. 2024. URL: <https://www.w3.org/TR/css-text-3/> (siehe S. 3-3).
- [25] World Wide Web Consortium. *CSS Transforms Module Level 2*. W3C Working Draft. 9. Nov. 2021. URL: <https://www.w3.org/TR/css-transforms-2/> (siehe S. 3-29).
- [26] World Wide Web Consortium. *CSS Transitions*. W3C Working Draft. 11. Okt. 2018. URL: <https://www.w3.org/TR/css-transitions-1/> (siehe S. 3-3).
- [27] World Wide Web Consortium. *Media Queries Level 3*. W3C Recommendation. 21. Mai 2024. URL: <https://www.w3.org/TR/mediaqueries-3/> (siehe S. 3-2).
- [28] World Wide Web Consortium. *Selectors Level 3*. W3C Recommendation. 6. Nov. 2018. URL: <https://www.w3.org/TR/selectors-3/> (siehe S. 3-2).

- [29] World Wide Web Consortium. *WOFF File Format 2.0*. W3C Recommendation. 8. Aug. 2024. URL: <https://www.w3.org/TR/WOFF2/> (siehe S. 3-33).